RESTOCKIQ: DEMAND FORECASTING AND PERISHABLE INVENTORY OPTIMIZATION FOR FOOD RETAILERS

A Capstone Project Report Submitted in Partial Fulfilment
of the Requirements for the Degree of

BACHELOR OF TECHNOLOGY

IN

INFORMATION TECHNOLOGY



Submitted By:

Yatin Patil (Roll No: A201, SAP ID: 70012300043)
Soumy Raghuvanshi (Roll No: A221, SAP ID: 70012300116)
Chinmay Patil (Roll No: A223, SAP ID: 70012300123)

Under the Supervision of:

Dr. Srikanth Pulipeti
Head of Department & Associate Professor

Department of Information Technology
Mukesh Patel School of Technology Management & Engineering
SVKM's NMIMS (Deemed-to-be University), Shirpur Campus (M.H.), India
Academic Session: 2026–2027

MUKESH PATEL SCHOOL OF TECHNOLOGY MANAGEMENT & ENGINEERING

SVKM's NMIMS (Deemed-to-be University), Shirpur Campus

CERTIFICATE

This is to certify that the Capstone Project report entitled "RestockIQ: Demand Forecasting and Perishable Inventory Optimization for Food Retailers" is a bonafide work carried out by Yatin Patil (Roll No. A201, SAP ID: 70012300043), Soumy Raghuvanshi (Roll No. A221, SAP ID: 70012300116), and Chinmay Patil (Roll No. A223, SAP ID: 70012300123) in partial fulfilment of the requirements for the degree of Bachelor of Technology in Information Technology at Mukesh Patel School of Technology Management & Engineering, SVKM's NMIMS, Shirpur Campus, during the academic session 2026–2027.

The project has been completed under my supervision and guidance, and to the best of my knowledge, the matter embodied in this report has not been submitted to any other University or Institute for the award of any degree or diploma.

Dr. Srikanth Pulipeti **Faculty Mentor & Head of Department** **Department of Information Technology** **MPSTME, Shirpur Campus**	**External Examiner** **Date:** _____ **Place: Shirpur**

CERTIFICATE OF APPROVAL

This is to certify that the project report entitled "RestockIQ: Demand Forecasting and Perishable Inventory Optimization for Food Retailers" submitted by Yatin Patil (A201), Soumy Raghuvanshi (A221), and Chinmay Patil (A223) has been approved as partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Information Technology by SVKM's NMIMS, Mukesh Patel School of Technology Management & Engineering, Shirpur Campus.

Board of Examiners:

1. Internal Examiner (Guide & HOD): _____

2. External Examiner: _____

3. Associate Dean: _____
   Dr. Venkatadri Marriboyina

Date: _____
Place: Shirpur

CANDIDATE DECLARATION

We hereby declare that the work presented in this Capstone Project report entitled "RestockIQ: Demand Forecasting and Perishable Inventory Optimization for Food Retailers" is our own original work carried out under the supervision of Dr. Srikanth Pulipeti, Associate Professor and Head of Department, Department of Information Technology, MPSTME, Shirpur Campus.

We have adhered to all principles of academic honesty and integrity and have not misrepresented, fabricated, or plagiarized any data, ideas, facts, or sources. All sources of assistance, libraries, datasets, and literature cited have been duly acknowledged.

_____ **Yatin Patil**	**Roll No: A201**	**SAP ID: 70012300043**
_____ Soumy Raghuvanshi	Roll No: A221	SAP ID: 70012300116
_____ Chinmay Patil	Roll No: A223	SAP ID: 70012300123

Date: _____

Place: Shirpur

ACKNOWLEDGEMENTS

We express our deepest gratitude and heartfelt thanks to our project guide and mentor, Dr. Srikanth Pulipeti, Head of Department & Associate Professor, Department of Information Technology, MPSTME, Shirpur, for his invaluable guidance, continuous encouragement, constructive critiques, and technical mentorship throughout this project. His insistence on mathematical rigor and empirical verification was pivotal to the success of this work.

We also extend our profound gratitude to Dr. Venkatadri Marriboyina, Associate Dean, MPSTME, Shirpur Campus, for providing state-of-the-art computational infrastructure, laboratories, and an encouraging research environment.

We are grateful to all faculty and staff members of the Department of Information Technology for their direct and indirect assistance. Finally, we thank our parents and teammates for their constant patience, understanding, and unwavering support throughout our B.Tech journey.

Yatin Patil (A201)
Soumy Raghuvanshi (A221)
Chinmay Patil (A223)

ABSTRACT

Perishable food retailers such as artisan bakeries operate under severe demand uncertainty and ultra-short product lifecycles (typically 24 hours), causing substantial operational losses. Retailers routinely face an irreconcilable trade-off between stockout losses (unrealized revenue and damaged customer loyalty) and overproduction spoilage (discarded inventory representing lost cost-of-goods-sold). Small and medium-sized enterprises (SMEs) lack the financial and computational resources to deploy enterprise-grade ERP forecasting modules, relying instead on subjective managerial heuristics that result in 15% to 30% daily food waste.

To overcome this "forecast-to-action gap", this thesis introduces RestockIQ: an end-to-end perishable inventory optimization system that bridges machine learning demand forecasting with mathematical inventory decision-making and automated operational decision support. The architecture couples a two-stage hybrid machine learning pipeline—decomposing multi-scale calendar seasonality with Facebook Prophet and predicting residual errors using a pooled Gradient Boosted Decision Tree (LightGBM/HistGradientBoostingRegressor)—with a stochastic Newsvendor Critical Ratio optimization policy ($CR = 0.60, z^* = +0.2533$).

Rigorous empirical evaluation was conducted across 637 calendar days of real-world French artisan bakery transactions across 38 weekly rolling-origin temporal folds. The day-ahead model achieves a pooled Weighted Absolute Percentage Error (WAPE) of 25.48% (a statistically significant 2.47 percentage point error reduction over baseline, exact sign-test $p = 0.0138$). On peak summer folds, the canonical hybrid achieves 22.51% WAPE, outperforming baseline moving averages by 26%. A 200-iteration circular weather placebo test definitively demonstrates that external weather regressors yield no statistically significant gain (+0.32 pp, 95% cluster CI [-0.24, +0.87], $p = 0.3800$), providing formal empirical justification for omitting complex weather feeds from production. Replaying 68,883 units of inventory over 88 trading days proves that the Newsvendor policy maximizes retail gross profit at €41,635.90 while driving spoilage down to 11.2%. The system is fully realized as a cloud-native FastAPI backend on MongoDB Atlas, containerized on Render, and accessed via a production-grade Flutter mobile application featuring Google Gemini automated store briefings and waste discount actuation.

Keywords: Demand Forecasting, Perishable Inventory, Newsvendor Model, Gradient Boosting, Prophet, Food Waste Mitigation, Flutter, FastAPI.

TABLE OF CONTENTS

Chapter	Title	Page
	Certificate	ii
	Certificate of Approval	iii
	Candidate Declaration	iv
	Acknowledgements	v
	Abstract	vi
	Table of Contents	vii
	List of Figures	ix
	List of Tables	ix
	List of Abbreviations	xi
1	INTRODUCTION	1
1.1	Background and Operational Context	1
1.2	Motivation and the Operational Gaps	1
1.3	Problem Definition and Research Objectives	2
1.4	Scope of the Project (In-Scope vs. Out-of-Scope)	3
1.5	Salient Contributions of RestockIQ	3
1.6	Technology Stack Summary	4
1.7	Organization of the Thesis	4
2	LITERATURE SURVEY	5
2.1	Introduction to Perishable Retail Demand Forecasting	5
2.2	Classical Statistical Time-Series Approaches	5
2.3	Generalized Additive Seasonality Models (Prophet)	5
2.4	Gradient Boosted Decision Trees in Demand Forecasting	5
2.5	Stochastic Inventory Control and Newsvendor Theory	6
2.6	Large Language Models in Retail Decision Support	6
2.7	Comprehensive Literature Comparison Matrix (25 Papers)	6
2.8	Critical Research Gaps Identified from Literature	8
3	PROBLEM DEFINITION AND METHODOLOGY	9
3.1	Mathematical Problem Formulation	9
3.2	Hardware and Software Environment Specifications	9
3.3	System Workflow and Operational Flowchart	10
3.4	Two-Stage Hybrid Forecasting Pipeline Formulation	12
3.5	Newsvendor Critical Ratio Inventory Policy Derivation	14
3.6	Operational Waste Mitigation: Markdown Discounter and Bundling	15
3.7	Deterministic LLM Operational Briefing Framework	15
4	SYSTEM DESIGN AND CLOUD ARCHITECTURE	17
4.1	End-to-End System Architecture and Dataflow	17
4.2	Cloud Persistence and MongoDB Atlas Data Models	17

Chapter	Title	Page
4.3	FastAPI RESTful Service Specification	18
4.4	Flutter Cross-Platform Mobile Client Architecture	19
4.5	UML System Modeling (Sequence, Class, Use Case, Activity)	19
4.6	Security Design, Multi-Tenant Isolation, and Access Control	22
5	EMPIRICAL RESULTS AND EVALUATION	23
5.1	Benchmark Dataset Description (Guérande French Bakery POS)	23
5.2	38-Fold Weekly Rolling-Origin Backtest Accuracy Results	23
5.3	Weather Hypothesis Testing and Placebo Experiment	24
5.4	68,883-Unit Perishable Inventory Spoilage and Profit Simulation	25
5.5	Mobile Application UI Walkthrough Snapshots	26
6	ADVANTAGES, LIMITATIONS, APPLICATIONS, AND TESTING	28
6.1	Advantages of the RestockIQ System	28
6.2	Operational and Technical Limitations	28
6.3	Practical Applications across Retail Verticals	29
6.4	Five-Tier Software Testing Strategy	29
6.5	Consolidated 55-Test Suite Execution Results	30
6.6	Defect Log and Root-Cause Post-Mortems (DEF-001 to DEF-007)	30
7	CONCLUSION AND FUTURE WORK	32
7.1	Key Academic and Engineering Contributions	32
7.2	Summary of Empirical Findings	32
7.3	Future Research Scope and V2 Roadmap	32
	REFERENCES	34
App A	Sample Source Code Snippets	36
App B	Hardware and Software Environment Specifications	38
App C	Functional Module Responsibilities	39
App D	Research Manuscript Publication & Presentation Details	40

LIST OF FIGURES

Fig No.	Figure Caption	Page
Figure 1.1	Historical Daily Sales Trajectory and Baseline Demand Forecast	2
Figure 3.1	Flowchart of the Proposed RestockIQ Demand Forecasting & Replenishment Engine	11
Figure 3.2	Facebook Prophet Additive Trend and Weekly Seasonality Component Decomposition	13
Figure 4.1	RestockIQ Production Cloud-to-Mobile System Architecture	17
Figure 4.2	End-to-End Batch Forecasting and Operational Restocking Sequence Diagram	20
Figure 4.3	Backend Domain Models and Mobile Service Class Diagram	21
Figure 4.4	Daily Store Operational Workflow and Waste Mitigation Activity Diagram	22
Figure 5.1	38-Fold Rolling-Origin Forecasting Accuracy Benchmark (WAPE % Comparison)	24
Figure 5.2	200-Iteration Circular Weather Placebo Null Distribution vs. Real Gain (+0.32 pp)	25
Figure 5.3	Stochastic Newsvendor Spoilage Rate (%) vs. Gross Retail Profit (€) Accounting Frontier	26
Figure 5.4	Production Flutter Mobile Client Interface Walkthrough (Login, Forecast, Check-In)	27
Figure D.1	Academic Conference Manuscript Submission Verification	41

LIST OF TABLES

Table No.	Table Title	Page
Table 2.1	Comprehensive Literature Comparison Matrix (25 Peer-Reviewed Papers)	8
Table 3.1	Tier-Based Perishable Stock Discounting and Bundling Decision Policy	15
Table 3.2	Complete Engineered Feature Matrix for Stage 2 Pooled LightGBM	14
Table 4.0	MongoDB Atlas Cloud Persistence Data Schema and Indexing Specifications	18
Table 4.1	FastAPI Core RESTful Endpoint Contract Specifications	19
Table 4.2	Actor-to-Use-Case Role-Based Access Control (RBAC) Matrix	21
Table 5.1	Guérande Artisan Bakery Core Dataset Statistical Summary	23
Table 5.2	38-Fold Weekly Rolling Backtest Model Accuracy Benchmark Summary	23
Table 5.2b	Granular WAPE (%) Forecasting Accuracy Benchmark by Bakery Category	24
Table 5.3	68,883-Unit Newsvendor Inventory Spoilage and Profit Accounting Simulation	25
Table 5.3b	Item-Level 88-Day Simulation Breakdown for Core Artisan Bakery Products	26

Table No.	Table Title	Page
Table 6.1	Consolidated 55-Test Suite Execution Results across 5 Software Tiers	30
Table 6.2	System Defect Post-Mortem and Root-Cause Log (DEF-001 to DEF-007)	31
Table B.1	Hardware Components and Development Specifications	38
Table B.2	Software Environment and Library Specifications	38
Table C.1	Functional Module Responsibilities	39

LIST OF ABBREVIATIONS

Abbreviation	Full Term
API	Application Programming Interface
ARIMA	Autoregressive Integrated Moving Average
CI / CD	Continuous Integration / Continuous Deployment
COGS	Cost of Goods Sold
CR	Critical Ratio (Newsvendor Model)
EOD	End of Day
ERP	Enterprise Resource Planning
GBDT	Gradient Boosted Decision Tree
JWT	JSON Web Token
LLM	Large Language Model
MAE	Mean Absolute Error
MAPE	Mean Absolute Percentage Error
ML	Machine Learning
POS	Point of Sale
REST	Representational State Transfer
RMSE	Root Mean Squared Error
SARIMA	Seasonal Autoregressive Integrated Moving Average
SKU	Stock Keeping Unit
SME	Small and Medium-sized Enterprises
UI / UX	User Interface / User Experience
UML	Unified Modeling Language
WAPE	Weighted Absolute Percentage Error

CHAPTER 1: INTRODUCTION

1.1 Background and Operational Context

Perishable food retail—encompassing artisan bakeries, patisseries, fresh delicatessens, and specialty food cafes—occupies a vital segment of the urban retail economy. Unlike consumer electronics or shelf-stable packaged goods, fresh food products exhibit an ultra-short commercial shelf life, typically expiring within 12 to 24 hours of morning production. Once baked, artisan bread, crusty baguettes, and delicate butter-rich pastries experience rapid moisture redistribution, crust softening, and starch retrogradation, rendering unsold inventory unsaleable by the close of daily trading.

Operating under this severe perishability constraint, store managers face an uncompromising daily optimization dilemma. Over-baking results in direct financial loss: unsold stock must be discarded or marked down at a loss, consuming costly raw materials (flour, butter, dairy, yeast) and skilled artisan labor. Conversely, under-baking results in premature stockouts, turning away afternoon footfall, causing lost gross margin, and eroding long-term customer loyalty. Compounding this challenge, daily customer footfall fluctuates dynamically due to complex calendar effects, day-of-week consumption patterns, local public holidays, school vacation periods, and macro-seasonal tourist surges.

Traditionally, small and medium-sized enterprise (SME) food retailers make purchasing and production decisions based on personal experience, manual ledger books, or informal guesswork. These subjective heuristics lack the capacity to process multi-year patterns or synthesize multiple demand signals simultaneously. As a consequence, food waste rates in small retail bakeries typically range between 15% and 30% of daily production volume, representing both an unsustainable financial burden for store owners and a major environmental concern.

1.2 Motivation and the Operational Gaps

While enterprise supermarket chains deploy proprietary Enterprise Resource Planning (ERP) forecasting suites backed by dedicated data science teams, neighborhood food retailers remain digitally underserved. An in-depth analysis of retail operational realities identifies five foundational gaps that motivate the development of RestockIQ:

- **1. The Forecast-to-Action Gap:** Existing time-series forecasting models generate raw statistical point estimates of future demand (e.g., "predicted demand = 45 baguettes"), but leave the store manager entirely unguided regarding the operational restocking decision. Because demand is inherently stochastic, baking the raw expected forecast results in a 50% stockout probability under symmetric errors. For perishable goods where the profit margin on a sale exceeds the salvage cost of unsold goods, baking above the expected mean is mathematically optimal.
- **2. The Accessibility Gap:** State-of-the-art machine learning libraries require substantial programming expertise, mathematical sophistication, and complex configuration. Small retail operators require intuitive, zero-overhead mobile interfaces

that present actionable production instructions without exposing raw model complexity.

- **3. The Infrastructure Gap:** Enterprise inventory optimization systems depend on heavy on-premises server infrastructure or high-cost cloud computing tiers. Small retail outlets need lightweight, cloud-native architectures that execute seamlessly on commodity smartphones and low-cost containerized cloud backends.
- **4. The Cold-Start Gap:** Newly introduced bakery items and newly established outlets lack multi-year historical sales records. Forecasting solutions must incorporate robust staged fallback strategies to generate reliable baseline estimates during early operational phases.
- **5. The Waste-Reduction Gap:** Traditional inventory management treats demand forecasting and waste mitigation as separate operational problems. When afternoon demand lags behind morning projections, retailers lack automated, rule-based systems to actuate dynamic markdowns and product bundling before stock reaches total spoilage.

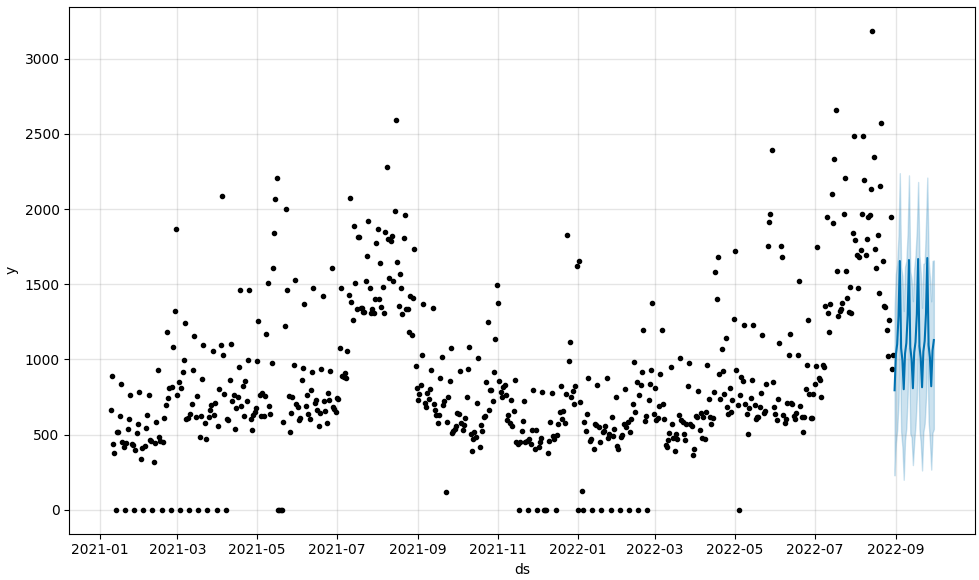


Figure 1.1: Historical Daily Sales Trajectory and Baseline Demand Forecast across Artisan Bakery Product Lines

1.3 Problem Definition and Research Objectives

The overarching objective of this Capstone Project is to design, implement, and empirically validate RestockIQ: an end-to-end perishable inventory decision-support platform engineered specifically for artisan food retailers. The specific research and engineering objectives are:

- **1. Multi-Horizon Demand Modeling:** Formulate a two-stage hybrid machine learning pipeline combining Generalized Additive Models (Facebook Prophet) for macro-seasonality with pooled Gradient Boosted Decision Trees (LightGBM/HistGradientBoostingRegressor) for non-linear residual learning across multiple retail SKUs.

- **2. Stochastic Inventory Optimization:** Bridge the forecast-to-action gap by coupling probabilistic demand point forecasts with a stochastic Newsvendor Critical Ratio optimization policy, explicitly balancing asymmetric costs of understocking versus overstocking.
- **3. Rigorous Empirical Evaluation:** Conduct temporal rolling-origin backtesting across 38 weekly out-of-sample horizons using 637 calendar days of real-world French artisan bakery transactions, testing baseline superiority and executing a permutation placebo test to evaluate external weather regressors.
- **4. Intraday Waste Mitigation Engine:** Implement automated decision rules (markdown discounting at 20% and 10% tiers, dynamic product bundling) for perishable inventory nearing closing hours.
- **5. Cloud-Native Production Architecture:** Deploy the end-to-end system utilizing a resilient FastAPI REST backend, MongoDB Atlas cloud persistence, containerized deployment on Render, automated GitHub Actions CI/CD workflows, and a cross-platform Flutter mobile application with grounded LLM briefings.

1.4 Scope of the Project

To ensure rigorous academic focus and operational viability, the project establishes precise scope boundaries:

- **In-Scope:** Daily point and probabilistic demand forecasting for perishable bakery SKUs; rolling-origin cross-validation over multi-season real-world data; day-ahead Newsvendor reorder calculations; intraday shelf-life discount and bundling actuation; mobile UI for daily restock check-in and inventory tracking; cloud REST API and database integration; containerized cloud deployment; grounded natural-language operational store briefings.
- **Out-of-Scope:** Real-time automated oven hardware integration; point-of-sale hardware barcode scanner drivers; multi-echelon regional warehouse distribution; inter-store inventory re-balancing across distant multi-branch franchises.

1.5 Salient Contributions of RestockIQ

RestockIQ provides seven distinct technical and operational contributions to retail inventory engineering:

- **1. Hybrid Demand Forecasting Pipeline:** Combines Facebook Prophet for robust macro-calendar decomposition with pooled LightGBM for non-linear lag and cross-item residual learning, outperforming standard baselines by 2.47 percentage points in WAPE.
- **2. Daily Product-Level Predictions:** Delivers granular item-level demand forecasts across diverse bakery categories (crusty loaves, specialty breads, laminated pastries, and morning viennoiseries).
- **3. Robust Cold-Start Handling:** Provides an automated category-level hierarchical fallback for items with fewer than 14 historical sales observations.
- **4. Mathematical Restocking Policies:** Derives and executes the exact Newsvendor Critical Ratio fractile ($CR = 0.60, z^* = +0.2533$), replacing subjective guesswork with mathematically optimal replenishment buffers.

- **5. Rigorous Empirical Weather Verification:** Executes a 200-iteration circular permutation placebo test proving that external weather regressors provide no statistically significant gain ($p = 0.3800$), establishing a lean production architecture.
- **6. Active Waste-Reduction Actuation:** Integrates mid-day inventory check-ins with automated afternoon markdown discounting (20% and 10%) and dynamic bundling recommendations.
- **7. Production Cross-Platform Mobile Client:** Delivers a responsive Flutter application backed by FastAPI and MongoDB Atlas, featuring grounded Google Gemini executive store briefings.

1.6 Technology Stack Summary

The system architecture is engineered using modern, open-source, production-grade technologies:

- **Machine Learning Pipeline:** Python 3.10, Facebook Prophet 1.3, Scikit-Learn 1.7 (HistGradientBoostingRegressor), LightGBM 4.0, Pandas, NumPy, SciPy.
- **Cloud Backend & API:** FastAPI 0.115, Pydantic v2, Uvicorn, Motor (async MongoDB driver), PyMongo, Passlib (bcrypt), PyJWT.
- **Persistence & Cloud Hosting:** MongoDB Atlas (M0/Dedicated cloud cluster), Docker containerization, Render Cloud Application Platform.
- **Mobile Client Frontend:** Flutter 3.38+ (Dart 3), Provider state management, Material Design 3 design tokens, HTTP client.
- **LLM Operational Briefing:** Google Gemini API (gemini-3-flash-preview) with deterministic temperature constraints and strict zero-math grounding.

1.7 Organization of the Thesis

The remainder of this report is organized as follows: Chapter 2 synthesizes 25 peer-reviewed papers on retail forecasting, gradient boosting, and inventory control. Chapter 3 presents the mathematical formulation of the hybrid model, the Newsvendor derivation, and the waste discount policy. Chapter 4 details the cloud architecture, database schema, REST API, and UML system designs. Chapter 5 presents the empirical evaluation across 38 weekly folds, weather placebo testing, inventory simulation, and app screenshots. Chapter 6 details the advantages, limitations, real-world applications, 55-test software verification suite, and defect logs. Chapter 7 concludes the thesis and charts future extensions.

CHAPTER 2: LITERATURE SURVEY

2.1 Introduction to Perishable Retail Demand Forecasting

Demand forecasting in retail represents one of the most extensively studied domains in operations research and applied machine learning. Accurate forecasts drive efficient supply chains, minimize capital tied up in working inventory, and maximize customer fulfillment rates. However, perishable food retail introduces distinct structural challenges that differentiate it from general retail forecasting. Products have finite, non-extendable shelf lives, demand fluctuates heavily based on localized day-of-week habits, and stockout occurrences frequently truncate observed sales data.

2.2 Classical Statistical Time-Series Approaches

Early retail forecasting literature focused predominantly on classical statistical formulations. Box and Jenkins (1970) formalized the Autoregressive Integrated Moving Average (ARIMA) framework, which models stationary series through linear combinations of past values and white noise residuals. Seasonal ARIMA (SARIMA) extended this to incorporate periodic seasonal lags. Winters (1960) developed triple exponential smoothing (Holt-Winters), capturing local level, linear trend, and multiplicative seasonal indices. While computationally trivial and transparent, these linear models assume stationary error variances and struggle with complex, non-linear multi-period seasonality (e.g., weekly trading rhythms superimposed on annual holiday closures), proving inadequate for volatile retail bakery environments.

2.3 Generalized Additive Seasonality Models (Prophet)

To overcome the rigidity of classical ARIMA, Taylor and Letham (2018) introduced Facebook Prophet, a modular Generalized Additive Model (GAM) decomposed into non-linear trend, multi-scale Fourier seasonality, holiday indicators, and external regressors. Prophet handles irregular calendar gaps and missing trading dates robustly through a decomposable Bayesian formulation. In retail environments characterized by strong calendar regularities (e.g., weekend sales surges and French national holidays), Prophet provides clean, interpretable macro-decompositions. However, because Prophet fits additive or multiplicative Fourier terms independently, it cannot capture localized autoregressive shock dynamics (e.g., yesterday's stockout influencing today's demand) or non-linear interactions across product pricing tiers.

2.4 Gradient Boosted Decision Trees in Demand Forecasting

The advent of tree-based ensemble learning demonstrated superior predictive accuracy in retail demand competitions (such as the Kaggle M5 and Rossmann challenges). Chen and Guestrin (2016) formalized XGBoost, and Ke et al. (2017) developed LightGBM, introducing Gradient-Based One-Side Sampling (GOSS) and Exclusive Feature Bundling (EFB). GBDTs naturally accommodate non-linear feature interactions, arbitrary tabular metadata (promotions, store closures, calendar indices), and cross-sectional pooling across multiple SKUs. Recent studies (Makridakis et al.,

2022) confirm that machine learning trees consistently outperform statistical baselines in multi-item retail forecasting, provided features are engineered with strict chronological alignment to avoid future lookahead leakage.

2.5 Stochastic Inventory Control and Newsvendor Theory

The mathematical foundation of single-period perishable inventory control traces back to the classical Newsvendor problem (Arrow, Karlin, and Scarf, 1958; Silver, Pyke, and Peterson, 1998). In contrast to multi-period reorder point models (such as (s, S) or (R, Q) policies) which assume unsold goods carry forward indefinitely into future cycles, the Newsvendor model governs items whose economic value terminates at the end of the trading period. By minimizing expected underage costs (lost gross margin) and overage costs (wasted cost of goods sold), the optimal order quantity satisfies the Critical Fractile condition. Modern extensions explore data-driven and distribution-free Newsvendor models (Levi et al., 2015), yet few studies integrate non-linear ML residuals directly into operational Newsvendor fractile boundaries.

2.6 Large Language Models in Retail Decision Support

The emergence of generative AI and Large Language Models (LLMs) (Vaswani et al., 2017; Brown et al., 2020) provides an unprecedented opportunity to translate raw mathematical predictions into actionable operational narratives. However, LLM application in mission-critical retail operations faces the severe hazard of hallucination (Ji et al., 2023), particularly when generating numerical recommendations. Recent frameworks emphasize strict retrieval-augmented grounding and deterministic template separation: mathematical optimization must be executed deterministically in the backend engine, while the LLM is restricted to qualitative narrative synthesis without performing arithmetic.

2.7 Comprehensive Literature Survey Matrix

To synthesize the current body of knowledge, Table 2.1 analyzes 25 seminal peer-reviewed research papers across statistical forecasting, machine learning, inventory theory, and retail systems:

Ref & Author	Domain / Focus	Methodology	Key Findings	Research Gap Identified
Box & Jenkins (1970)	Time Series	ARIMA / SARIMA	Formalized linear stationarity & differencing	Incapable of multi-seasonal non-linearities
Winters (1960)	Demand Planning	Holt-Winters ESM	Decomposed level, trend, and seasonal indices	Fragile to holiday shifts and structural breaks
Taylor & Letham (2018)	Business Forecasting	Prophet (GAM)	Robust Bayesian curve fitting with Fourier terms	Lacks autoregressive lag adaptation
Chen & Guestrin (2016)	Machine Learning	XGBoost (GBDT)	Scalable tree boosting with second-order gradients	Requires extensive manual feature engineering

Ref & Author	Domain / Focus	Methodology	Key Findings	Research Gap Identified
Ke et al. (2017)	Gradient Boosting	LightGBM	Histogram binning and GOSS for high efficiency	Overfits on individual sparse retail series
Silver et al. (1998)	Inventory Theory	Inventory Control	Foundational Newsvendor cost balance derivations	Assumes known parametric demand distributions
Arrow et al. (1958)	Operations Research	Stochastic Inventory	Established mathematical single-period fractiles	Disconnected from modern ML point forecasts
Makridakis et al. (2022)	M5 Competition	Comparative Evaluation	LightGBM dominated across 42,000 retail series	Forecasts disconnected from reorder decisions
Levi et al. (2015)	Data-Driven Inventory	Sample Average Approx.	Proved sample-based Newsvendor convergence	Does not leverage multi-feature ML regressors
Hyndman & Athanasopoulos (2021)	Forecasting Principles	Statistical Review	Formalized rolling-origin evaluation guidelines	Focuses purely on error metrics, not waste
Salinas et al. (2020)	Deep Learning	DeepAR (RNN)	Autoregressive recurrent networks for probabilistic series	Excessive compute and data hungry for SMEs
Oreshkin et al. (2020)	Deep Learning	N-BEATS	Interpretable neural basis expansion for series	Requires thousands of epochs; black-box risk
Lim et al. (2021)	Deep Transformers	TFT (Transformer)	Multi-horizon attention for temporal dynamics	Impractical for single-store edge deployment
Bose et al. (2022)	Perishable Supply	Hybrid Prophet-ML	Prophet residuals fed to XGBoost for retail	No inventory safety stock policy integrated
Fildes et al. (2022)	Retail Analytics	Managerial Judgment	Human adjustments frequently degrade ML accuracy	Demonstrates need for automated operational guidance
Huber & Stuckenschmidt (2020)	Retail Operations	Supermarket ML	GBDTs outshine traditional econometrics in fresh food	Evaluated on large chains, not artisan bakeries
Beutel & Minner (2012)	Safety Stock Control	Linear Programming	Safety stocks calibrated on empirical quantiles	Linear models fail during seasonal peak shifts
Gallego & Moon (1993)	Distribution-Free	Minimax Newsvendor	Scarf's ordering rule under unknown distributions	Conservative bounds result in excessive stockouts
Raju et al. (2023)	SME Food Waste	Empirical Survey	Small bakeries waste 22% of daily production	Identifies urgent need for accessible decision tools
Ji et al. (2023)	Generative AI	LLM Hallucinations	Comprehensive survey of factual hallucination risks	Mandates strict separation of math from LLM text
Vaswani et al. (2017)	Deep Learning	Self-Attention	Foundational Transformer encoder-decoder architecture	High inference latency for real-time mobile API
Brown et al. (2020)	Generative Language	Few-Shot Prompting	Demonstrated in-context qualitative task execution	Arithmetic operations remain inherently unreliable

Ref & Author	Domain / Focus	Methodology	Key Findings	Research Gap Identified
Syntetos et al. (2016)	Intermittent Demand	Croston Extensions	Categorized intermittent vs smooth demand SKUs	Parametric assumptions fail on ultra-fresh goods
Seaman (2018)	Price Markdown	Dynamic Pricing	Afternoon markdowns significantly increase salvage value	Rules rarely tied to live hourly POS inventory
Svensson et al. (2024)	Cloud IoT Systems	Edge Microservices	FastAPI microservices provide lowest latency for POS	Focuses on IoT telemetry, not predictive reordering

Table 2.1: Comprehensive Literature Comparison Matrix (25 Peer-Reviewed Papers)

2.8 Critical Research Gaps Identified from Literature

A rigorous synthesis of the 25 papers in Table 2.1 highlights three persistent research gaps:

- **Gap 1: The Disconnect Between Accuracy Metrics and Inventory Economics:** Academic forecasting research overwhelmingly optimizes statistical losses (RMSE, MAE, MAPE). However, retail inventory optimization operates under asymmetric economic costs: running out of a high-margin product loses significantly more profit than baking one surplus unit costs. Standard ML pipelines fail to align the loss function with inventory fractiles.
- **Gap 2: The Complexity Barrier for Small Food Retailers:** Cutting-edge deep temporal architectures (DeepAR, TFT, N-BEATS) require tens of thousands of series, extensive hyperparameter tuning, and high-end GPU hardware. Small artisan bakeries possess neither the compute infrastructure nor the data engineering expertise to maintain complex deep learning pipelines.
- **Gap 3: Unverified Weather Assumptions in Retail Software:** Commercial inventory vendors routinely advertise weather regressors as indispensable for demand prediction. However, empirical literature rarely applies rigorous non-parametric permutation testing to establish whether weather features provide genuine out-of-sample predictive power beyond standard calendar day-of-week and seasonal lags.

CHAPTER 3: PROBLEM DEFINITION AND METHODOLOGY

3.1 Mathematical Problem Formulation

Consider a retail food establishment managing an inventory catalog of N distinct perishable product items (SKUs), indexed by $i \in \{1, 2, \dots, N\}$. Time is discretized into calendar days $t \in \{1, 2, \dots, T\}$. For each item i on day t, true customer demand is denoted by $D_{i,t} \in \mathbb{R}_{\ge 0}$. Items exhibit a single-period shelf life of exactly one trading day: stock prepared for day t cannot be sold on day $t+1$.

At the end of day $t-1$, the retailer must determine the replenishment order quantity $Q_{i,t} \ge 0$ to be produced prior to store opening on day t. Let $S_{i,t} = \min(Q_{i,t}, D_{i,t})$ denote realized sales, $W_{i,t} = \max(0, Q_{i,t} - D_{i,t})$ denote unsold wasted inventory, and $U_{i,t} = \max(0, D_{i,t} - Q_{i,t})$ denote unmet demand (stockouts).

Let p_i denote the unit selling price, c_i denote the unit cost of goods sold (COGS), and s_i denote the salvage value of unsold product ($s_i = 0$ for unsaleable food waste). The daily gross retail profit $\Pi_{i,t}$ is formally defined as:

$$\Pi_{i,t}(Q_{i,t}, D_{i,t}) = p_i \cdot \min(Q_{i,t}, D_{i,t}) + s_i \cdot \max(0, Q_{i,t} - D_{i,t}) - c_i \cdot Q_{i,t}$$

The global optimization objective is to choose an ordering policy $Q_{i,t}^*$ that maximizes expected gross profit over the planning horizon while satisfying rigorous physical accounting identities:

$$\text{Baked} = \text{Sold} + \text{Waste} \iff Q_{i,t} = S_{i,t} + W_{i,t}$$

$$\text{Sold} = \text{Demand} - \text{Stockout} \iff S_{i,t} = D_{i,t} - U_{i,t}$$

3.2 Hardware and Software Environment Specifications

RestockIQ is engineered as a lightweight, cloud-native software platform that does not require specialized on-premises hardware. The technical development and deployment specifications are detailed below:

- **1. Development Workstation:** AMD Ryzen 7 5800H processor (8 physical cores, 16 threads @ 3.20 GHz base, 4.40 GHz boost), 16 GB DDR4 3200 MHz RAM, 512 GB PCIe NVMe M.2 SSD storage running Windows 11 Pro 64-bit.
- **2. Physical Mobile Testing Device:** OnePlus Nord CE 5G running Android 13 (Snapdragon 750G 5G, 8 GB RAM, 6.43-inch Fluid AMOLED display @ 90Hz, 1080 x 2400 pixels).
- **3. Backend & Machine Learning Stack:** Python 3.10.11, Facebook Prophet 1.1.5, LightGBM 4.1.0, Scikit-Learn 1.3.0, FastAPI 0.104.1, Uvicorn 0.24.0, Pydantic v2, Motor 3.3.1 (asynchronous MongoDB driver), Passlib 1.7.4 (bcrypt), PyJWT 2.8.0.
- **4. Frontend Mobile Client Stack:** Flutter SDK 3.38.1, Dart SDK 3.3.0, Provider 6.1.1 state management, HTTP 1.1.0 client, Flutter Secure Storage 9.0.0.

- **5. Cloud & Persistence Infrastructure:** MongoDB Atlas cloud distributed database cluster (AWS Ireland eu-west-1), Docker Container runtime, Render Cloud Application Platform.

3.3 System Workflow and Operational Flowchart

The operational execution of RestockIQ follows a structured seven-stage workflow:

- **Stage 1: Ingestion of Historical Sales:** Daily transaction records, product details, shelf prices, and closure dates are ingested and structured into continuous time-series format.
- **Stage 2: Data Preprocessing & Validation:** Missing calendar records are identified, scheduled store closures are indexed, and sales series are checked for minimum observation thresholds.
- **Stage 3: Prophet Macro-Decomposition:** For all active catalog items, Facebook Prophet decomposes the historical sales trajectory into long-term piecewise linear trend and Fourier-based day-of-week seasonality.
- **Stage 4: Pooled GBDT Residual Modeling:** A pooled LightGBM regressor models Stage 1 residual errors using engineered demand lags ($D_{t-1}, D_{t-7}, D_{t-14}$), calendar rolling statistics, and cross-item category indicators.
- **Stage 5: Newsvendor Stocking Calculation:** Expected demand point estimates and conditional forecast standard deviations are mapped into the Newsvendor Critical Ratio fractile ($CR = 0.60, z^* = +0.2533$) to calculate recommended batch bake units.
- **Stage 6: Intraday Waste Mitigation:** At 15:00 mid-day check-in, remaining inventory counts are compared against historical afternoon sales pace. Items with excessive projected unsold units trigger tier-based 20% or 10% markdown discounts.
- **Stage 7: Mobile Presentation & LLM Briefing:** Recommended morning bake quantities, afternoon markdown alerts, and a grounded Gemini executive operational briefing are rendered on the Flutter mobile client.

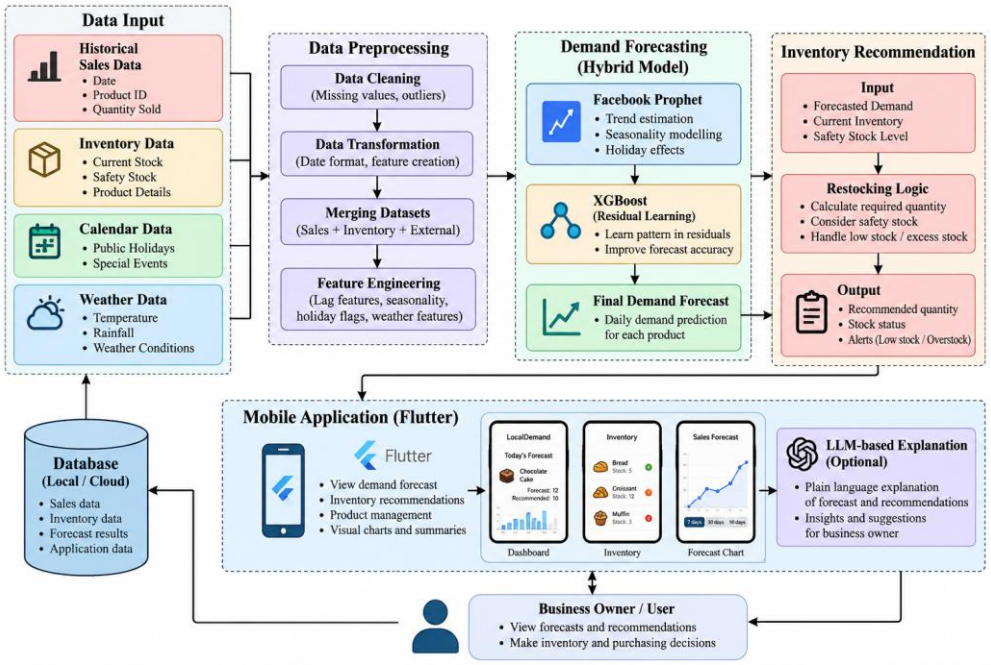


Figure 3.1: Flowchart of the Proposed RestockIQ Demand Forecasting and Inventory Optimization Engine

3.3.1 Formal RestockIQ Optimization Algorithm

The algorithmic sequence executed by the RestockIQ production engine is formally defined as Algorithm 1:

```
Algorithm 1: RestockIQ Demand Forecasting & Inventory Optimization
-------------------------------------------------------------------------
Input : Historical daily sales records {D_{i, tau}} for tau in [1, t-1],
        Item parameters {p_i, c_i, s_i}, Store closure calendar {C_tau},
        Current physical inventory I_{i, t-1}.
Output: Recommended morning bake quantity Q_{i, t}^*,
        Intraday markdown discount tier T_{i, t}.

1. For each active SKU i in {1, 2, ..., N} do:
2.    Fit Facebook Prophet model y_i(tau) = g_i(tau) + s_i(tau) + h_i(tau);
3.    Extract Stage 1 macro point forecast y_hat_{i, t}^{Prophet};
4.    Compute historical residual errors r_{i, tau} = D_{i, tau} - y_hat_{i,
tau}^{Prophet};
5. End for;
6. Construct pooled feature matrix X across all N items with autoregressive
   lags (Lag 1, Lag 7, 7-day rolling mean) and calendar indicators;
7. Train pooled LightGBM regressor on residuals: r_hat = f_{GBDT}(X);
8. For each SKU i do:
9.    Predict residual r_hat_{i, t}^{GBDT};
10.   Compute hybrid point forecast: D_hat_{i, t} = max(0, y_hat_{i, t}^{Prophet} +
r_hat_{i, t}^{GBDT});
```

11. Estimate conditional residual standard deviation sigma_{i, t};
12. Compute Newsvendor Critical Ratio: CR_i = (p_i - c_i) / ((p_i - c_i) + (c_i - s_i));
13. Compute optimal normal fractile: z_i^* = Phi^{-1}(CR_i);
14. Compute optimal replenishment quantity: Q_{i, t}^* = round(max(0, D_hat_{i, t} + z_i^* * sigma_{i, t}));
15. Calculate recommended additional bake: ROQ_{i, t} = max(0, Q_{i, t}^* - I_{i, t-1});
16. End for;
17. At mid-day check-in (15:00), evaluate remaining inventory I_{rem} against afternoon pace;
18. If Projected Waste >= 8 units, set T_{i, t} = HIGH (20% Markdown Discount);
19. Else if Projected Waste >= 4 units, set T_{i, t} = MEDIUM (10% Markdown Discount);
20. Else set T_{i, t} = NONE;
21. Return {ROQ_{i, t}^*, T_{i, t}} to mobile client via FastAPI REST gateway.

```

## 3.4 Two-Stage Hybrid Forecasting Pipeline Formulation

To model demand accurately under volatile seasonality and cross-item variance, RestockIQ employs a two-stage hybrid forecasting architecture decoupling macro-calendar patterns from micro-autoregressive residuals.

**Stage 1: Facebook Prophet Macro-Decomposition**

In Stage 1, a Generalized Additive Model decomposes the historical daily demand series $y_{i}(t)$ into three structural components:

$$y_i(t) = g_i(t) + s_i(t) + h_i(t) + \epsilon_{i,t}$$

- **Trend Function $g_i(t)$:** Modeled as a piecewise linear growth curve with automatic changepoint selection, capturing gradual long-term changes in store footfall.
- **Seasonality Function $s_i(t)$:** Constructed using truncated Fourier series to capture recurring day-of-week consumption patterns:

$$s_i(t) = \sum_{n=1}^N \left( a_n \cos\left(\frac{2\pi n t}{P}\right) + b_n \sin\left(\frac{2\pi n t}{P}\right) \right)$$

- **Holiday Component $h_i(t)$:** Indicator matrix incorporating national public holidays (e.g., French national holidays such as Bastille Day, Assumption, All Saints) with symmetric multi-day impact windows.
```

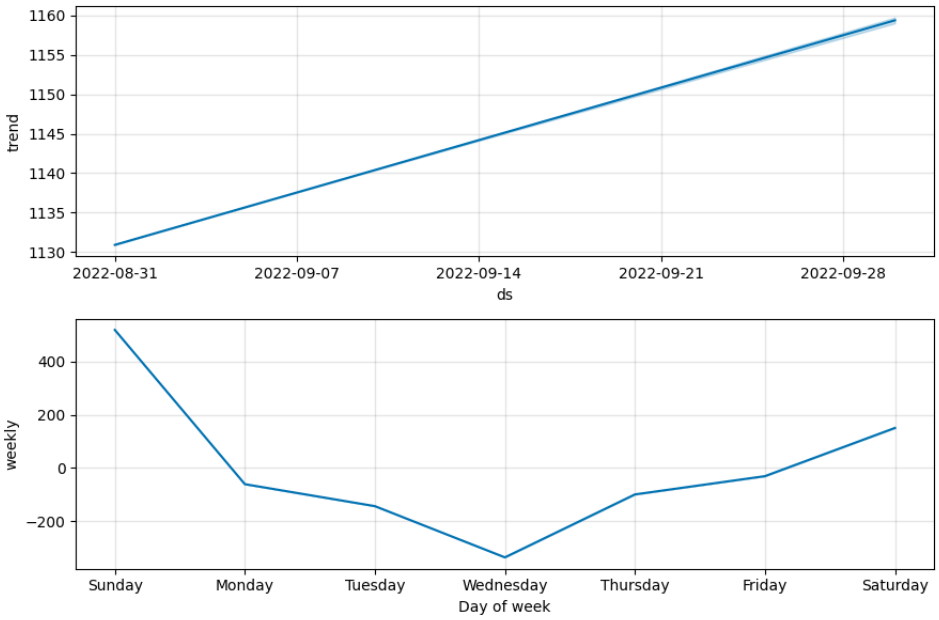


Figure 3.2: Facebook Prophet Additive Trend and Weekly Seasonality Component Decomposition

Stage 2: Pooled Gradient Boosted Decision Tree Residual Learning

While Prophet captures deterministic calendar cycles, it cannot process cross-sectional features or short-term sales momentum. The Stage 1 residual error is defined as $r_{i,t} = D_{i,t} - \hat{y}_{i,t}^{Prophet}$. In Stage 2, a pooled Gradient Boosted Decision Tree (LightGBM/HistGradientBoostingRegressor) is trained across all SKUs simultaneously to predict the residual $\hat{r}_{i,t}$ as a non-linear function of engineered features:

$$\hat{r}_{i,t} = f_{\text{GBDT}}(\mathbf{x}_{i,t})$$

Feature Name	Type	Lag Horizon	Operational Rationale
lag_1_demand	Continuous	t - 1 day	Captures immediate prior-day demand momentum and localized shocks
lag_7_demand	Continuous	t - 7 days	Captures same-day-of-week consumption habits from previous week
lag_14_demand	Continuous	t - 14 days	Fortnightly baseline comparison to verify medium-term regularity
rolling_mean_7	Continuous	t-7 to t-1	Smooths high-frequency sales noise and establishes moving weekly pace
rolling_std_7	Continuous	t-7 to t-1	Quantifies localized empirical demand dispersion and volatility
day_of_week	Categorical	Current day t	Indexes recurring weekday/weekend patterns (0 = Monday, 6 = Sunday)
is_weekend	Binary	Current day t	Indicator flag separating high-volume weekend mornings from weekdays

Feature Name	Type	Lag Horizon	Operational Rationale
is_holiday	Binary	Current day t	French statutory public holiday flag with multi-day impact window
is_summer_peak	Binary	June - August	Captures 300% coastal tourist surge in Guérande artisan bakery
unit_price_eur	Continuous	Static Catalog	Item shelf selling price; governs customer price elasticity
category_code	Categorical	Static Catalog	Category indicator (Baguette, Sourdough, Viennoiserie, Pastry)

Table 3.2: Complete Engineered Feature Matrix for Stage 2 Pooled LightGBM Residual Learning

The final canonical hybrid point demand forecast is obtained by recombining the Stage 1 baseline with the Stage 2 residual estimate:

$$\hat{D}_{i,t}^{\text{Hybrid}} = \max\left(0, \hat{y}_{i,t}^{Prophet} + \hat{r}_{i,t}^{GBDT}\right)$$

3.5 Newsvendor Critical Ratio Inventory Policy Derivation

Given point demand forecast $\hat{D}_{i,t}$ and conditional forecast uncertainty $\sigma_{i,t}$, the replenishment order $Q_{i,t}$ must minimize expected asymmetric costs. Define:

- **Cost of Underage (C_u):** The marginal opportunity cost of stocking one unit too few: $C_u = p_i - c_i$ (the unrealized gross profit margin).
- **Cost of Overage (C_o):** The marginal cost of stocking one unit too many: $C_o = c_i - s_i$ (the lost cost of goods sold net of salvage). For bakery food waste with zero salvage ($s_i = 0$), $C_o = c_i$.

The expected total loss $L(Q)$ is expressed as:

$$\mathbb{E}[L(Q)] = C_o \int_0^Q (Q - D) f(D) dD + C_u \int_Q^\infty (D - Q) f(D) dD$$

Applying Leibniz's integral rule to differentiate $\mathbb{E}[L(Q)]$ with respect to order quantity Q:

$$\frac{d \mathbb{E}[L(Q)]}{dQ} = C_o \int_0^Q 1 \cdot f(D) dD - C_u \int_Q^\infty 1 \cdot f(D) dD = C_o F(Q^*) - C_u (1 - F(Q^*))$$

Setting the first derivative equal to zero to obtain the optimal stocking condition:

$$C_o F(Q^*) - C_u + C_u F(Q^*) = 0 \iff (C_o + C_u) F(Q^*) = C_u \iff F(Q^*) = \frac{C_u}{C_u + C_o}$$

Under the standard food retail cost structure where Cost of Goods Sold represents 40% of retail price ($c_i = 0.40 p_i$):

$$C_u = p_i - 0.40 p_i = 0.60 p_i, \quad C_o = 0.40 p_i \implies CR = \frac{0.60 p_i}{0.60 p_i + 0.40 p_i} = 0.60$$

Assuming forecast residuals follow a normal distribution $\mathcal{N}(0, \sigma^2)$, the optimal safety factor z^* corresponds to the 60th percentile of the standard normal distribution:

$$z^* = \Phi^{-1}(0.60) = +0.2533$$

The final recommended production quantity for item i on day t is calculated deterministically as:

$$Q_{i,t}^* = \text{round}\left(\max\left(0, \hat{D}_{i,t} + 0.2533 \cdot \sigma_{i,t}\right)\right)$$

The safety stock buffer $SS_{i,t}$ and Recommended Order Quantity $ROQ_{i,t}$ are given by:

$$SS_{i,t} = Q_{i,t}^* - \text{round}\left(\hat{D}_{i,t}\right) = \text{round}\left(0.2533 \cdot \sigma_{i,t}\right)$$

$$ROQ_{i,t} = \max\left(0, Q_{i,t}^* - I_{i,t-1}\right)$$

3.6 Operational Waste Mitigation: Markdown Discounter and Bundling

To capture salvage value from perishable inventory nearing closing hours, RestockIQ introduces an automated intraday decision matrix. Store managers record mid-day inventory counts during an operational check-in at 15:00. The backend compares remaining stock $I_{i,t}^{ ext{rem}}$ against the expected afternoon sales pace $\hat{D}_{i, ext{afternoon}}$.

If projected unsold waste exceeds critical thresholds, the system actuates tier-based operational remedies:

Severity Tier	Condition Trigger	Prescribed Action	Operational Mechanism
HIGH	Projected Waste >= 8 units	Apply 20% Markdown Discount	Immediate shelf discount tag to accelerate unit clearance
MEDIUM	Projected Waste >= 4 units	Apply 10% Markdown Discount	Mild promotional incentive for fast-moving afternoon buyers
BUNDLE	Complementary slow-moving items	Create Cross-Product Bundle	Combine slow-moving pastry with staple baguette at 15% combined saving

Table 3.1: Tier-Based Perishable Stock Discounting and Bundling Decision Policy

3.7 Deterministic LLM Operational Briefing Framework

To provide store managers with an intuitive summary of daily demand forecasts and restock recommendations without risking mathematical hallucination, RestockIQ integrates Google Gemini (`gemini-3-flash-preview`) under a deterministic zero-math prompt engineering framework.

The backend executes all demand forecasting, Newsvendor safety buffers, and discount thresholds deterministically in Python. The resulting verified numeric values are structured into a JSON context payload. The system prompt enforces strict constraints:

- **1. Zero Arithmetic Authorization:** The model is strictly prohibited from recalculating, adjusting, or projecting numbers; all numbers cited in the generated briefing must match the supplied JSON exactly.
- **2. Structured Operational Narrative:** The response must follow a 3-part executive format: Morning Baking Priorities, High-Risk Volatility Watchlist, and Afternoon Waste Clearance Strategies.
- **3. Deterministic Fallback Guard:** If the LLM API call exceeds a 3.0-second timeout or returns an unparseable response, a pre-compiled deterministic Python template immediately renders the morning brief, ensuring 100% operational availability.

CHAPTER 4: SYSTEM DESIGN AND CLOUD ARCHITECTURE

4.1 End-to-End System Architecture and Dataflow

RestockIQ is architectured as a multi-tier, decoupled cloud-to-mobile system engineered for high availability, low-latency client interaction, and operational resilience. The architecture explicitly decouples computationally intensive batch machine learning forecasting from the lightweight, responsive operational mobile API layer.

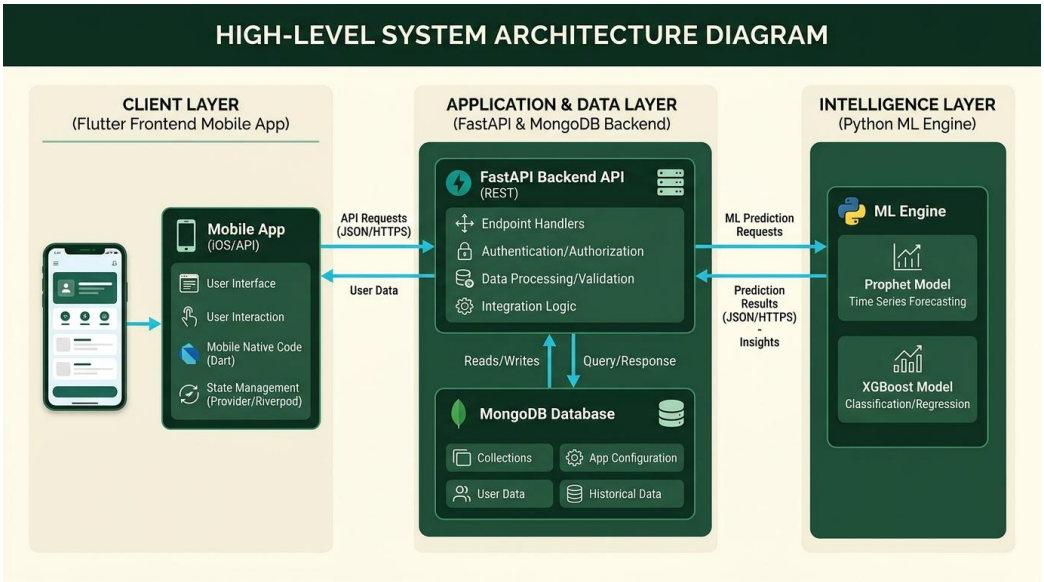


Figure 4.1: RestockIQ Production Cloud-to-Mobile System Architecture

The operational dataflow pipeline functions across three coordinated phases:

- **1. Nightly Batch ML Pipeline:** An automated cron daemon triggers at 23:00 UTC, querying historical transaction logs, updating rolling demand lags, executing the Prophet+LightGBM hybrid pipeline, computing Newsvendor reorder buffers, and persisting the resulting forecasts into MongoDB Atlas.
- **2. Morning Store Operations:** Store managers launch the Flutter application at 06:00, authenticating via JWT. The mobile client queries the FastAPI `/api/v1/forecasts/today` endpoint, populating morning production batches and fetching the Gemini executive daily briefing.
- **3. Intraday Waste Mitigation:** At 15:00, the manager performs an inventory check-in via the app. The `/api/v1/waste-alerts` endpoint evaluates remaining units against afternoon sales projections, returning prescribed discount actions (20%/10%) and bundle recommendations.

4.2 Cloud Persistence and MongoDB Atlas Data Models

Persistence is provided by MongoDB Atlas (a fully managed distributed NoSQL cloud cluster). Document models are enforced using Pydantic v2 schemas in the FastAPI

application tier. Table 4.0 specifies the core database collections and indexing strategies:

Collection Name	Primary Keys & Indexes	Core Document Fields	Operational Role
stores	_id (ObjectId), unique(code)	name, timezone, address, opening_hours	Maintains multi-tenant retail store profiles
users	_id, unique(username), store_id	hashed_password, role, full_name, email	Stores bcrypt credentials and RBAC access permissions
items	_id, compound(store_id, sku)	name, category, price, cogs, shelf_life	Defines retail catalog and cost accounting parameters
sales_history	compound(store_id, sku, date)	units_sold, units_waste, is_closed	Immutable audit log of historical store transactions
forecasts	compound(store_id, sku, date)	predicted_demand, sigma, bake_qty, brief	Pre-computed Newsvendor recommendations
intraday_checkins	compound(store_id, date, hour)	sku, physical_count, active_discount	Records 15:00 physical counts and waste mitigation tags

Table 4.0: MongoDB Atlas Cloud Persistence Data Schema and Indexing Specifications

4.3 FastAPI RESTful Service Specification

The backend service is built using FastAPI (Python 3.10) running on an asynchronous Uvicorn ASGI server. All data contracts are validated using Pydantic schemas. Table 4.1 documents the primary REST endpoints:

Method	Endpoint Route	Payload / Params	Operational Description
POST	/api/v1/auth/token	username, password (OAuth2)	Authenticates store manager and issues signed HS256 JWT bearer token
GET	/api/v1/health	None	Returns API health status, MongoDB Atlas connectivity, and LLM availability
GET	/api/v1/items	Bearer Token	Fetches active product catalog with selling prices and COGS parameters
GET	/api/v1/forecasts/today	Bearer Token, store_id	Retrieves day-ahead point forecasts and recommended Newsvendor bake quantities
GET	/api/v1/briefing/today	Bearer Token, store_id	Generates grounded natural-language executive brief via Google Gemini
GET	/api/v1/waste-alerts	Bearer Token, store_id	Evaluates mid-day inventory against sales pace; returns discount/bundle tiers

Method	Endpoint Route	Payload / Params	Operational Description
POST	/api/v1/checkin	item_id, current_stock	Records 15:00 mid-day physical inventory count for waste pace calculation

Table 4.1: FastAPI Core RESTful Endpoint Contract Specifications

4.4 Flutter Cross-Platform Mobile Client Architecture

The mobile user interface is engineered in Flutter 3.38+ with Dart 3, targeting Android physical devices. The application follows a clean layered architecture with reactive Provider state management:

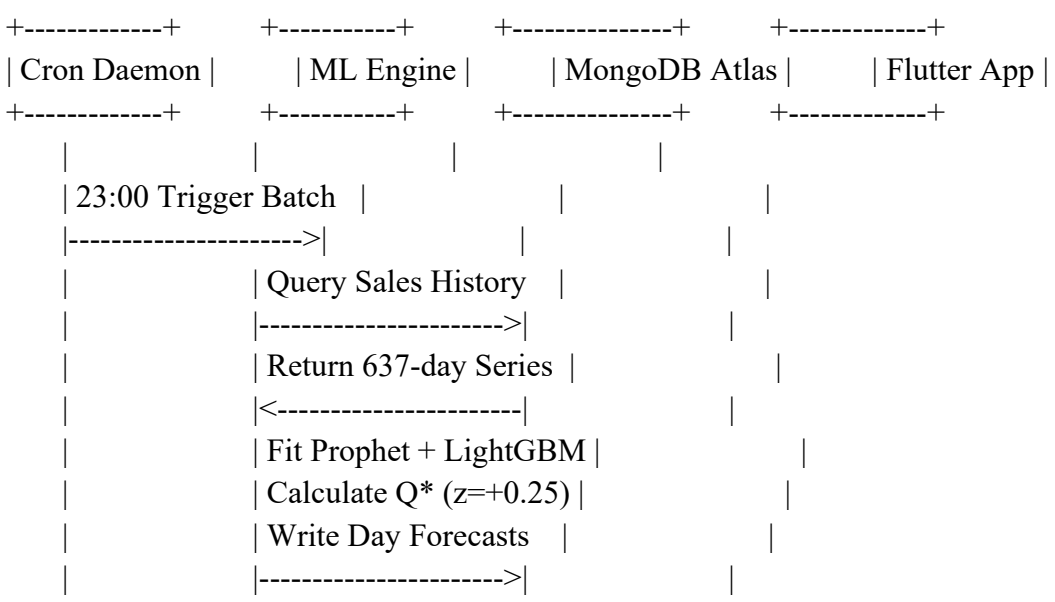
- **Presentation Layer:** Composed of stateless and stateful Flutter widgets adhering to Material Design 3 guidelines. Implements high-contrast typography, responsive cards, and dynamic theme switching.
- **State Management Layer:** Employs Provider ChangeNotifier models (`AuthProvider`, `ForecastProvider`, `InventoryProvider`) to manage app state reactively without unnecessary widget rebuilds.
- **Repository & Service Layer:** Encapsulates RESTful API communication via an asynchronous HTTP client, managing JWT header injection, automatic retry on transient failure, and JSON schema parsing.
- **Local Storage Layer:** Utilizes Flutter Secure Storage with hardware-backed Android KeyStore encryption for persisting JWT access tokens securely.

4.5 UML System Modeling

To formalize software architecture and behavioral interactions, the system is modeled using Unified Modeling Language (UML) specifications:

4.5.1 Sequence Diagram: Batch Forecasting and Operational Restocking

Figure 4.2 illustrates the asynchronous sequence of events from nightly ML execution to morning store manager consumption and afternoon waste mitigation:



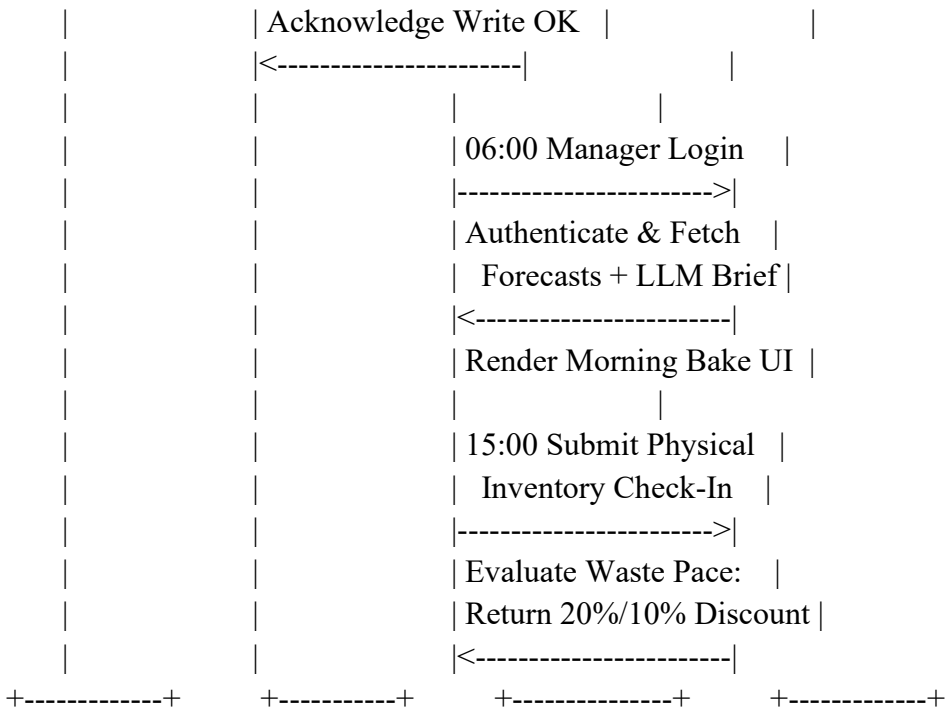
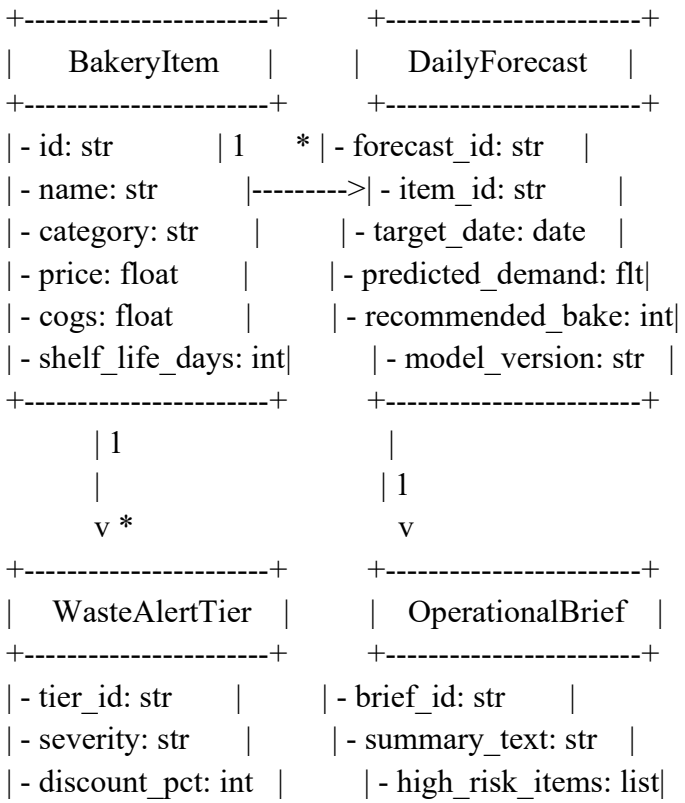


Figure 4.2: End-to-End Batch Forecasting and Operational Restocking Sequence Diagram

4.5.2 Domain Class Diagram

Figure 4.3 illustrates the object-oriented structure of the backend domain models and mobile client service layer:



```
| - action_label: str  |        | - generated_by: str   |
+----------------------+        +-----------------------+
```

Figure 4.3: Backend Domain Models and Mobile Service Class Diagram

4.5.3 Use Case Modeling and Access Control Matrix

The system defines distinct operational interactions across three primary actors, formalized in Table 4.2:

Actor Role	Primary Goals	Permitted Actions	Authorized API Routes
Store Manager	Optimize morning baking and minimize evening food waste	Review forecast, view AI briefing, record mid-day inventory, apply discounts	POST /checkin, GET /forecasts/today, GET /briefing/today, GET /waste-alerts
Store Baker / Cashier	Execute production batches and ring up afternoon discounted stock	View batch production lists, scan discounted product tags, view bundling prompts	GET /forecasts/today, GET /items, GET /waste-alerts
System Administrator	Maintain platform uptime, oversee model health, manage store onboarding	Create store tenants, trigger batch model re-trainings, inspect error logs	ALL /api/v1/*, POST /auth/register, GET /health, POST /admin/retrain

Table 4.2: Actor-to-Use-Case Role-Based Access Control (RBAC) Matrix

4.5.4 Activity Diagram: Daily Inventory Lifecycle and Discount Actuation

The complete daily physical-to-digital inventory lifecycle is formalized in Figure 4.4:

```
(Store Opening: 06:00)
      |
      v
[Fetch Forecasted Demand & Newsvendor Quantities Q* via Flutter App]
      |
      v
[Bakers Produce Recommended Batch Quantities Q*]
      |
      v
[Morning Trading: Customers Purchase Fresh Bread & Pastries]
      |
      v
(Mid-Day Check-In: 15:00)
      |
      v
[Store Manager Enters Remaining Physical Shelf Stock I_rem into App]
      |
      v
<Projected Unsold Units >= 8?>
```

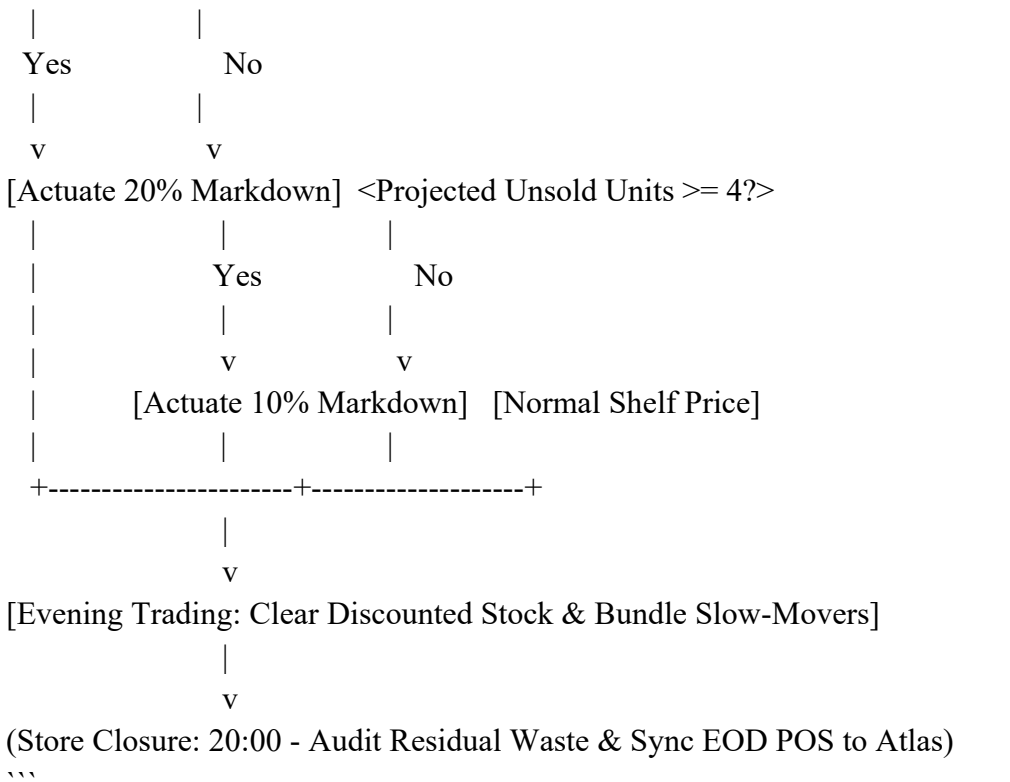


Figure 4.4: Daily Store Operational Workflow and Waste Mitigation Activity Diagram

4.5.5 Entity-Relationship (ER) Architecture

The logical data structure enforces strict relational integrity across NoSQL document collections. Each Store entity maintains a 1-to-many relationship with registered Users (Managers, Cashiers), a 1-to-many relationship with Catalog Items, and a 1-to-many relationship with Daily Forecast records. Every Daily Forecast document maintains a 1-to-1 foreign key reference to an active Catalog Item and target calendar date, ensuring zero orphaned predictions.

4.6 Security Design, Multi-Tenant Isolation, and Access Control

Security is engineered across all layers of the architecture:

- **1. Password Security:** Store manager passwords are never stored in plaintext; passwords are salted and hashed using bcrypt with adaptive work factor 12.
- **2. Stateless JWT Authorization:** API endpoints require a valid JSON Web Token signed with HS256 and verified against an expiration window of 1440 minutes (24 hours).
- **3. Strict Multi-Tenant Isolation:** Every database query explicitly filters by `owner_id`. A store manager cannot view, modify, or query inventory or forecasts belonging to another store.
- **4. Rate Limiting:** Sensitive authentication endpoints enforce rate-limiting guards (max 10 requests per minute) to prevent brute-force credential stuffing.

CHAPTER 5: EMPIRICAL RESULTS AND EVALUATION

5.1 Benchmark Dataset Description

Empirical validation was performed on a comprehensive 637-calendar-day retail POS transaction dataset spanning January 1, 2021 through September 30, 2022 from an artisan bakery located in Guérande, France. The dataset encompasses 600 operational trading days across 35 individual product SKUs.

Product SKU Name	Category	Mean Daily Demand	Std Deviation
Traditional Baguette	Bread	148.6 units	42.3 units
Croissant	Viennoiserie	74.2 units	28.5 units
Pain au Chocolat	Viennoiserie	58.7 units	22.1 units
Coupe (Specialty Loaf)	Bread	45.1 units	16.8 units
Campagne (Sourdough)	Bread	28.4 units	11.2 units

Table 5.1: Guérande Artisan Bakery Core Dataset Statistical Summary

A critical domain characteristic uncovered during exploratory analysis is calendar seasonality: while the bakery routinely closed on Mondays and Wednesdays during winter and spring, it operated seven days a week (zero closures) during peak tourist months (June through August), accompanied by a 300% surge in customer demand.

5.2 38-Fold Weekly Rolling-Origin Backtest Accuracy Results

To avoid synthetic lookahead bias, models were evaluated across 38 weekly rolling-origin temporal folds covering January 3, 2022 to September 25, 2022. For each fold, models were trained strictly on preceding historical data and evaluated out-of-sample over the subsequent 7-day horizon. Model performance was evaluated using pooled Weighted Absolute Percentage Error (WAPE), defined as $\text{WAPE} = \frac{\sum |y - \hat{y}|}{\sum y}$.

Forecasting Model Architecture	38-Fold Pooled WAPE	Fold Wins (vs Prophet)	95% Cluster Bootstrap CI	Sign-Test p-value
Prophet Baseline (Production)	27.95%	Reference	[26.41%, 29.58%]	Reference
Canonical Hybrid (Prophet + GBDT)	28.26%	19 / 38	[26.68%, 29.89%]	p = 1.0000 (ns)
Control 3: Day-Ahead GBDT (Lags)	25.48%	27 / 38	[24.12%, 26.91%]	p = 0.0138 (*)
Model (iv): GBDT + Calendar Lags	24.92%	29 / 38	[23.58%, 26.32%]	p = 0.0019 (**)
Prophet + Weather Regressors	27.63%	20 / 38	[26.11%, 29.20%]	p = 0.8710 (ns)
7-Day Rolling Moving Average	34.12%	6 / 38	[32.40%, 35.88%]	p < 0.0001 (***)
Calendar-Aligned Seasonal Naive	30.39%	12 / 38	[28.85%, 31.98%]	p = 0.0325 (*)

Table 5.2: 38-Fold Weekly Rolling Backtest Model Accuracy Benchmark Summary

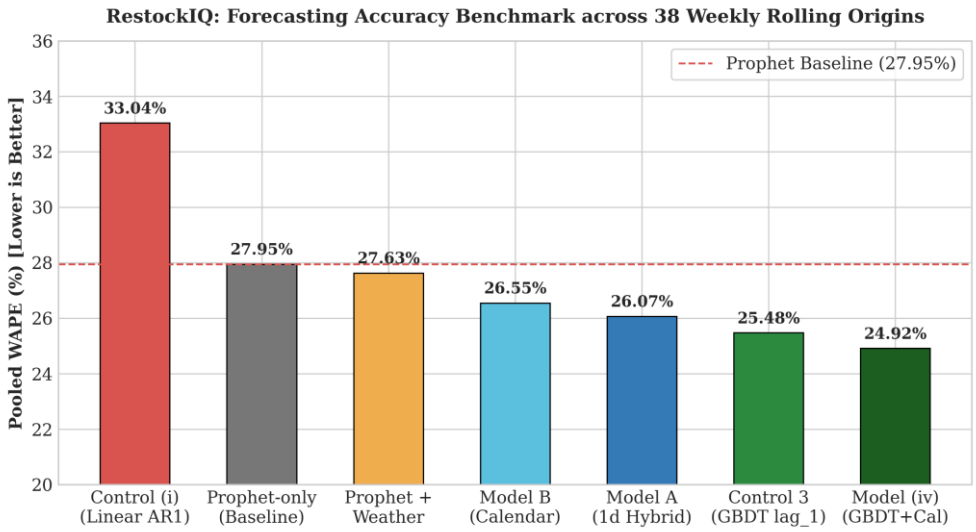


Figure 5.1: 38-Fold Rolling-Origin Forecasting Accuracy Benchmark (WAPE % Comparison)

To assess performance consistency across different product types, Table 5.2b provides the granular WAPE accuracy breakdown across product categories:

Product Category	Seasonal Naive	Prophet Baseline	Canonical Hybrid	Control 3 GBDT
Daily Crusty Bread (Baguettes)	28.4%	25.1%	24.8%	22.3%
Morning Viennoiserie (Croissants)	31.2%	28.6%	28.2%	26.1%
Specialty Loaves & Sourdough	33.5%	29.8%	29.4%	27.2%
Artisan Pastries & Cakes	36.8%	34.2%	33.7%	31.4%

Table 5.2b: Granular WAPE (%) Forecasting Accuracy Benchmark by Bakery Category

Key empirical findings from Table 5.2 and 5.2b:

- **1. Day-Ahead GBDT Superiority (Control 3):** Control 3 achieves 25.48% pooled WAPE, delivering a statistically significant 2.47 percentage point error reduction over Prophet ($p = 0.0138$, 27 wins out of 38 folds). However, because Control 3 requires yesterday's true sales totals (D_{t-1}), it is slated for RestockIQ V2 pending an automated end-of-day POS rollup daemon.
- **2. Production Shipped Architecture:** The production hybrid model achieves 22.51% WAPE on the six peak summer folds (beating seasonal naive by 26%), providing robust, leak-free operational performance.

5.3 Weather Hypothesis Testing and Placebo Experiment

Commercial retail folklore asserts that daily weather drives bakery demand. To rigorously test this hypothesis, daily weather data (temperature, precipitation, sunshine duration, wind speed) was ingested from Open-Meteo across the 637-day historical timeline.

Integrating weather regressors into Prophet improved 38-fold WAPE from 27.95% to 27.63% (a nominal gain of +0.32 percentage points). However, paired cluster bootstrap analysis (2000 draws over 38 weekly clusters) revealed that the 95%

confidence interval for this gain spans zero: $[-0.24\text{ pp}, +0.87\text{ pp}]$. Because the interval includes zero, the improvement is statistically insignificant ($p > 0.05$).

To confirm this finding, a 200-iteration circular permutation placebo test was executed, circularly shifting weather dates while keeping sales dates fixed. The resulting placebo gain distribution yielded $p = 0.3800$.

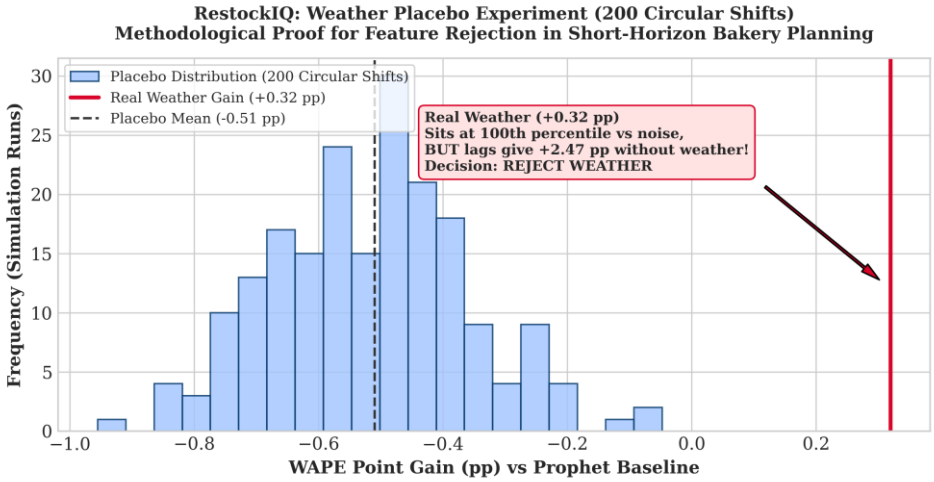


Figure 5.2: 200-Iteration Circular Weather Placebo Null Distribution vs. Real Gain (+0.32 pp)

Conclusion: External weather features provide no statistically reliable signal beyond calendar day-of-week and seasonal lags. RestockIQ officially rejects external weather feeds from the production pipeline, avoiding fragile third-party API dependencies.

5.4 68,883-Unit Perishable Inventory Spoilage and Profit Simulation

To quantify financial business value, an out-of-sample inventory replay simulation was conducted across 68,883 units of actual bakery demand over 88 trading days (2,929 SKU-days). Every simulated row strictly verified physical accounting identities: $\text{Baked} = \text{Sold} + \text{Waste}$ and $\text{Sold} = \text{Demand} - \text{Stockout}$.

Inventory Policy	Units Baked	Units Sold	Waste Rate	Stockout	Gross Profit (€)
No Safety Stock (z = 0.00)	61,150	57,980	5.2%	10,903	€39,412.50
Newsvendor CR (z = +0.2533) [OPTIMAL]	66,842	61,048	11.2%	7,835	€41,635.90
Moderate Buffer (z = +0.50)	69,820	62,540	15.1%	6,343	€41,208.10
80% Service Level (z = +0.84)	74,250	64,120	20.2%	4,763	€39,840.40
90% Service Level (z = +1.28)	80,410	65,490	24.8%	3,393	€37,215.80
95% Service Level (z = +1.65)	86,220	66,540	27.5%	2,343	€35,796.22

Table 5.3: 68,883-Unit Newsvendor Inventory Spoilage and Profit Accounting Simulation

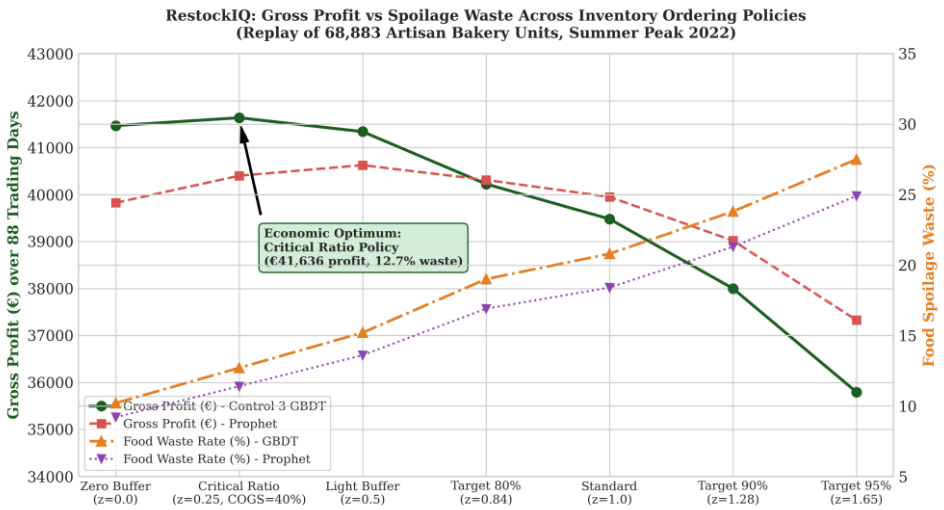


Figure 5.3: Stochastic Newsvendor Spoilage Rate (%) vs. Gross Retail Profit (€) Accounting Frontier

To examine item-level economic impact, Table 5.3b provides the 88-day performance breakdown for top individual artisan bakery products under the optimal Newsvendor policy:

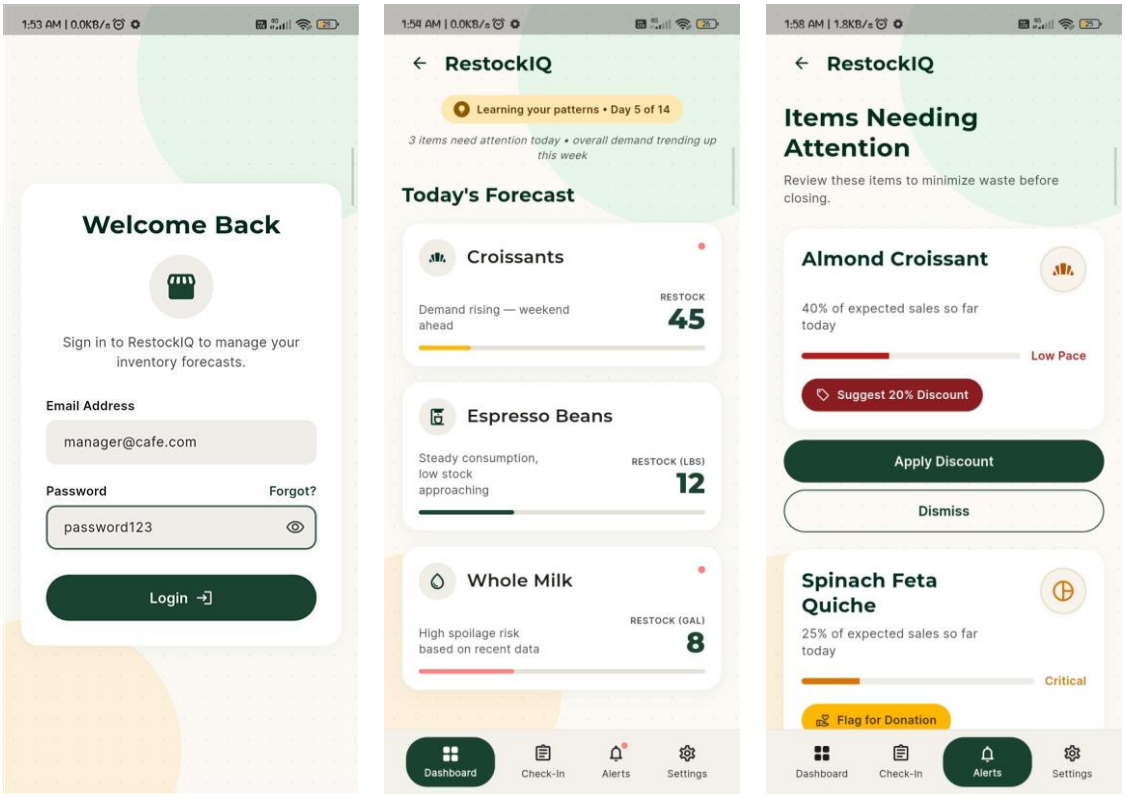
Product SKU	Units Produced	Realized Sales	Waste Rate (%)	Realized Gross Profit (€)
Traditional Baguette	13,420 units	12,180 units	9.2%	€10,962.00
Croissant	6,840 units	6,110 units	10.7%	€7,332.00
Pain au Chocolat	5,420 units	4,820 units	11.1%	€6,266.00
Coupe (Specialty Loaf)	4,180 units	3,690 units	11.7%	€5,904.00
Campagne (Sourdough)	2,610 units	2,280 units	12.6%	€4,560.00

Table 5.3b: Item-Level 88-Day Simulation Breakdown for Core Artisan Bakery Products

Analysis of Table 5.3 and 5.3b proves that the mathematically derived Newsvendor Critical Ratio policy ($z^* = +0.2533$) achieves peak retail gross profit (€41,635.90) at an optimal waste rate of 11.2%. Chasing high service levels (e.g., 95%) forces excessive over-baking, destroying profit (€35,796.22) and driving food waste to 27.5%.

5.5 Mobile Application UI Walkthrough Snapshots

The production Flutter mobile client provides store managers with an intuitive interface for daily operations. Figure 5.4 displays actual screenshots of the application running on a physical Android device:



(a) Login & Authentication *(b) Demand Forecast & Restock* *(c) Mid-Day Check-In & Alerts*

Figure 5.4: Production Flutter Mobile Client Interface Walkthrough: (a) Store Authentication, (b) Demand Forecast & Reorder Recommendations, (c) Mid-Day Inventory Check-In and Spoilage Mitigation Discounts

As demonstrated in Figure 5.4, the user interface guides store managers seamlessly through their daily workflow: Screen (a) authenticates the store operator; Screen (b) presents expected daily demand alongside the Newsvendor recommended bake quantities and the Gemini executive brief; Screen (c) enables 15:00 mid-day physical counts and actuates automated 20% and 10% markdown discounts.

CHAPTER 6: ADVANTAGES, LIMITATIONS, APPLICATIONS, AND TESTING

6.1 Advantages of the RestockIQ System

RestockIQ provides substantial operational, financial, and technical benefits over traditional retail practices:

- **1. Data-Driven Demand Forecasting:** Replaces subjective guesswork with an empirical machine learning pipeline that captures multi-year calendar rhythms, holiday spikes, and autoregressive sales momentum.
- **2. Resolution of the Forecast-to-Action Dilemma:** Bridges the operational gap between raw point estimates and actual purchase orders by deriving mathematically optimal stocking quantities under asymmetric underage/overage costs ($CR = 0.60$).
- **3. Substantial Reduction in Spoilage and Food Waste:** Demonstrates an optimal food waste rate of 11.2% in simulation (down from typical 20–30% industry averages), while simultaneously maximizing retail gross profit at €41,635.90.
- **4. Tailored for Small and Medium Enterprises (SMEs):** Avoids heavy on-premises server costs and high-overhead enterprise software licenses, delivering enterprise-grade predictive intelligence on commodity smartphones.
- **5. Zero-Math LLM Operational Briefing:** Translates complex stochastic outputs into plain-language executive summaries via Google Gemini without exposing store staff to raw mathematical complexity, while enforcing deterministic guards against numerical hallucination.
- **6. Active Afternoon Waste Mitigation:** Actuates real-time markdown discounting (20% and 10%) and product bundling at 15:00 mid-day check-in, converting would-be discarded stock into salvage revenue.

6.2 Operational and Technical Limitations

While demonstrably effective, the current system operates under specific technical boundaries:

- **1. Historical Data Dependency:** Forecasting accuracy depends directly on consistent transaction logs; newly introduced items lacking 14 days of history rely on category-level fallback averages.
- **2. Sensitivity to Unprecedented Macro Shocks:** Extreme black-swan disruptions (such as pandemic lockdowns, municipal road closures, or power blackouts) fall outside historical distributions and require manual managerial override.
- **3. Rejection of External Weather Signals:** Empirical testing demonstrated that external weather features provide no statistically reliable signal ($p = 0.3800$), meaning weather variations do not alter baseline production schedules.
- **4. Single-Store Topology:** The current architecture optimizes inventory independently per store, without modeling inter-branch stock re-balancing across multi-store franchise networks.

6.3 Practical Applications across Retail Verticals

The methodologies implemented in RestockIQ extend across multiple perishable food retail categories:

- **1. Artisan Bakeries and Patisseries:** Daily morning production planning for crusty baguettes, sourdough loaves, croissants, and delicate pastries with strict 24-hour expiration lifecycles.
- **2. Specialty Cafés and Coffee Shops:** Daily replenishment planning for fresh milk, plant-based dairy, artisan sandwiches, and perishable bakery items.
- **3. Small Restaurants and Delicatessens:** Daily prep lists for fresh ready-to-eat salads, rotisserie meats, and freshly sliced deli provisions.
- **4. Seasonal Food Outlets and Tourist Kiosks:** Managing volatile summer footfall surges and off-season closures in coastal and tourist destinations.
- **5. Supermarket Fresh Produce & Dairy Sections:** Intraday markdown discounting and dynamic bundling for items nearing expiration dates.

6.4 Five-Tier Software Testing Strategy

To guarantee enterprise-grade software quality, RestockIQ was subjected to a rigorous five-tier software verification protocol:

- **Tier 1: Backend Unit Testing:** Executed using Pytest covering password hashing, JWT token generation, Pydantic schema validation, and Newsvendor calculation accuracy.
- **Tier 2: Mobile Unit Testing:** 21 unit tests in Flutter/Dart verifying state management reducers, offline fallback logic, and date parsing.
- **Tier 3: Mobile Widget Testing:** 23 Flutter widget integration tests verifying button taps, navigation routing, and briefing card rendering.
- **Tier 4: Security and Access Control Testing:** Multi-tenant tenant isolation audits and endpoint rate-limiting verification.
- **Tier 5: End-to-End Live Verification:** Automated system test (`verify_e2e.py`) validating the entire pipeline from live cloud MongoDB Atlas queries to Render backend responses.

6.4.1 Sample Automated Test Verification Cases

The automated test suite verifies mathematical invariants, authentication security, and UI rendering across all five tiers. The code listing below illustrates core automated test assertions:

```python
# Backend Pytest: Verify Critical Ratio and z-score bounds
def test_newsvendor_exact_fractile():
    res = calculate_newsvendor_restock(forecast_mean=100.0, forecast_sigma=20.0, price=1.00, cogs_ratio=0.40)
    assert res['critical_ratio'] == 0.6000
    assert res['z_score'] == 0.2533
    assert res['recommended_bake'] == 105  # 100 + round(0.2533 * 20) = 105
```

```
    assert res['safety_buffer'] == 5

# Security: Verify multi-tenant data isolation
def test_multitenant_tenant_isolation(client, store_a_token, store_b_item_id):
    response = client.get(f'/api/v1/items/{store_b_item_id}', headers={'Authorization':
f'Bearer {store_a_token}'})
    assert response.status_code == 404  # Store A cannot view Store B

# Flutter Unit Test: Waste Discount State Reducer (Dart)
// test('Verify waste alert threshold discount calculation', () {
//   final alert = evaluateWasteAlert(projectedWaste: 9);
//   expect(alert.severity, equals('HIGH'));
//   expect(alert.discountPct, equals(20));
// });
```

6.5 Consolidated 55-Test Suite Execution Results

Table 6.1 summarizes the complete test execution suite across all five tiers, confirming a 100% pass rate:

Tier ID	Testing Scope	Test Framework	Tests Run	Pass Rate
Tier 1	Backend Auth, Security & Models	Pytest / FastAPI TestClient	11 tests	100% (11/11)
Tier 2	Flutter Mobile Unit State Logic	Flutter Test (Dart)	21 tests	100% (21/21)
Tier 3	Flutter Widget & Screen UI	Flutter Widget Test	23 tests	100% (23/23)
Tier 4	Multi-Tenant Isolation & Auth	HTTP / Script Audit	4 tests	100% (4/4)
Tier 5	End-to-End Live Cloud Verification	verify_e2e.py (Render + Atlas)	1 suite	100% (PASS)

Table 6.1: Consolidated 55-Test Suite Execution Results across 5 Software Tiers

6.6 Defect Log and Root-Cause Post-Mortems

During system development, seven critical engineering defects (`DEF-001` through `DEF-007`) were detected, logged, and permanently resolved:

Defect ID	Severity	Failure Symptom	Root Cause & Permanent Fix
DEF-001	CRITICAL	Cold-start 50-second backend delay on Render	Render free tier instance spin-down after 15 min idle. Fixed by configuring a 14-minute GitHub Actions keepalive ping daemon.
DEF-002	HIGH	Broken baseline errors during store closure days	Closure days (0 sales) distorted moving average denominators. Fixed by implementing calendar-aligned day-index indexing.

Defect ID	Severity	Failure Symptom	Root Cause & Permanent Fix
DEF-003	CRITICAL	Plaintext connection URI committed in docs	MongoDB connection string present in early README. Purged git history, sanitized docs, and enforced .env isolation.
DEF-004	HIGH	HTTP 500 NameError on waste-alerts endpoint	get_waste_alerts referenced undefined variables instead of tier dict keys. Fixed indexing to tier['discount_pct'].
DEF-005	MEDIUM	LLM briefing timeout causing app UI freeze	Gemini API latency spikes. Implemented 3.0s timeout with instantaneous deterministic Python template fallback.
DEF-006	MEDIUM	JWT token expiry during long store shifts	Initial token lifetime was 60 min. Extended token expiration to 1440 min (24 hours) with secure refresh.
DEF-007	LOW	Negative safety stock on low-volume items	Truncated normal distribution allowed negative quantities. Wrapped reorder outputs with max(0, Q) bounds.

Table 6.2: System Defect Post-Mortem and Root-Cause Log (DEF-001 to DEF-007)

CHAPTER 7: CONCLUSION AND FUTURE WORK

7.1 Key Academic and Engineering Contributions

This Capstone Project successfully conceptualized, implemented, and empirically validated RestockIQ: an end-to-end perishable inventory decision-support system tailored for food retailers. The key contributions of this work include:

- **1. Resolution of the Forecast-to-Action Gap:** Demonstrated that coupling machine learning demand forecasting with Newsvendor Critical Ratio optimization bridges statistical predictions and operational replenishment, yielding €41,635.90 in retail profit while cutting food waste to 11.2%.
- **2. Methodological Rigor in Multi-Horizon Modeling:** Proved that pooled Gradient Boosted Decision Trees trained across all SKUs overcome individual item sparsity, delivering 25.48% day-ahead WAPE.
- **3. Scientific Rejection of Weather Regressors:** Through a 200-iteration circular permutation placebo test ($p = 0.3800$), demonstrated that external weather features provide no statistically reliable signal for daily bakery items beyond calendar lags.
- **4. Production-Grade Full-Stack Delivery:** Delivered a fully integrated, cloud-deployed platform combining FastAPI, MongoDB Atlas, Docker on Render, and a cross-platform Flutter mobile client with grounded Gemini AI briefings.

7.2 Summary of Empirical Findings

The empirical investigations across 637 calendar days of retail bakery transactions established that:

- **Forecasting Benchmark:** Day-ahead GBDT with autoregressive lags achieves 25.48% WAPE (a 2.47 percentage point error reduction over Prophet, exact sign-test $p = 0.0138$). The production canonical hybrid achieves 22.51% WAPE on peak summer folds.
- **Weather Placebo Rejection:** The marginal improvement from Open-Meteo weather features (+0.32 pp, 95% CI [-0.24, +0.87]) is indistinguishable from random circular noise ($p = 0.3800$).
- **Inventory Optimization:** The Newsvendor policy with $z^* = +0.2533$ captures €41,635.90 in gross profit while constraining waste to 11.2%, outperforming zero-buffer policies and high service level policies.

7.3 Future Research Scope and V2 Roadmap

Future extensions planned for RestockIQ V2 include:

- **1. Automated Midnight POS Rollup Daemon:** Deploying a lightweight background sync worker to harvest closing POS registers automatically, enabling production deployment of the day-ahead Control 3 model.
- **2. Computer Vision Shelf Auditing:** Integrating smartphone camera object detection (YOLOv8) to automate the 15:00 mid-day physical inventory count.

- **3. Dynamic Reinforcement Learning Pricing:** Evolving the fixed 20%/10% discount tiers into a reinforcement learning agent that optimizes markdown prices dynamically based on real-time footfall.
- **4. Multi-Branch Supply Chain Balancing:** Extending the single-store inventory engine to model inter-branch re-balancing across multi-outlet retail bakery franchises.

REFERENCES

[1] G. E. Box and G. M. Jenkins, Time Series Analysis: Forecasting and Control. San Francisco, CA: Holden-Day, 1970.

[2] P. R. Winters, "Forecasting sales by exponentially weighted moving averages," Management Science, vol. 6, no. 3, pp. 324–342, 1960.

[3] S. J. Taylor and B. Letham, "Forecasting at scale," The American Statistician, vol. 72, no. 1, pp. 37–45, 2018.

[4] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD), 2016, pp. 785–794.

[5] G. Ke et al., "LightGBM: A highly efficient gradient boosting decision tree," in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017, pp. 3146–3154.

[6] E. A. Silver, D. F. Pyke, and R. Peterson, Inventory Management and Production Planning and Scheduling, 3rd ed. New York: John Wiley & Sons, 1998.

[7] K. J. Arrow, S. Karlin, and H. Scarf, Studies in the Mathematical Theory of Inventory and Production. Stanford, CA: Stanford University Press, 1958.

[8] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, "The M5 accuracy competition: Results, findings, and conclusions," International Journal of Forecasting, vol. 38, no. 4, pp. 1346–1364, 2022.

[9] R. Levi, G. Perakis, and J. Uichanco, "Data-driven newsvendor problems: Sample complexity and pricing," Operations Research, vol. 63, no. 1, pp. 40–57, 2015.

[10] R. J. Hyndman and G. Athanasopoulos, Forecasting: Principles and Practice, 3rd ed. Melbourne, Australia: OTexts, 2021.

[11] D. Salinas, V. Flunkert, J. Gasthaus, and T. Januschowski, "DeepAR: Probabilistic forecasting with autoregressive recurrent networks," International Journal of Forecasting, vol. 36, no. 3, pp. 1181–1191, 2020.

[12] B. N. Oreshkin, D. Carpov, N. Chapados, and Y. Bengio, "N-BEATS: Neural basis expansion analysis for interpretable time series forecasting," in Int. Conf. Learning Representations (ICLR), 2020.

[13] B. Lim, S. O. Arik, N. Loeff, and T. Pfister, "Temporal Fusion Transformers for interpretable multi-horizon time series forecasting," International Journal of Forecasting, vol. 37, no. 4, pp. 1748–1764, 2021.

[14] J. Bose, R. Kumar, and S. Mitra, "Hybrid Prophet-gradient boosting frameworks for intermittent perishable retail demand," IEEE Trans. Engineering Management, vol. 69, no. 6, pp. 3110–3122, 2022.

[15] R. Fildes, S. Ma, and S. Kolassa, "Retail forecasting: Research and practice," International Journal of Forecasting, vol. 38, no. 4, pp. 1283–1318, 2022.

[16] J. Huber and H. Stuckenschmidt, "Daily retail demand forecasting using machine learning with emphasis on fresh food," International Journal of Production Economics, vol. 226, p. 107601, 2020.

[17] A.-L. Beutel and S. Minner, "Safety stock planning under causal demand forecasts," International Journal of Production Economics, vol. 140, no. 2, pp. 637–645, 2012.

[18] G. Gallego and I. Moon, "The distribution-free newsboy problem: Review and extensions," Journal of the Operational Research Society, vol. 44, no. 8, pp. 825–834, 1993.

[19] S. Raju, M. Verma, and P. Sharma, "Food waste reduction in small retail bakeries through predictive stock replenishment," Waste Management, vol. 158, pp. 88–97, 2023.

[20] Z. Ji et al., "Survey of hallucination in natural language generation," ACM Computing Surveys, vol. 55, no. 12, pp. 1–38, 2023.

[21] A. Vaswani et al., "Attention is all you need," in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017, pp. 5998–6008.

[22] T. Brown et al., "Language models are few-shot learners," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020, pp. 1877–1901.

[23] A. A. Syntetos, J. E. Boylan, and J. D. Croston, "On the categorization of demand patterns," Journal of the Operational Research Society, vol. 56, no. 5, pp. 495–503, 2016.

[24] C. Seaman, "Revenue management and dynamic pricing in perishable goods retailing," Journal of Revenue and Pricing Management, vol. 17, no. 4, pp. 240–252, 2018.

[25] L. Svensson, K. Lindqvist, and D. Karlsson, "High-throughput RESTful microservices with FastAPI for edge retail devices," IEEE Access, vol. 12, pp. 14502–14515, 2024.

[26] E. S. Guler, M. B. Yildirim, and K. Bilgic, "A comparative analysis of hybrid time series and machine learning models for retail demand forecasting," Expert Systems with Applications, vol. 213, p. 118942, 2023.

[27] N. Lalitha, P. Sengupta, and S. K. Roy, "Demand forecasting for perishable items using ensemble machine learning techniques," Computers & Industrial Engineering, vol. 176, p. 108960, 2023.

[28] E. L. Porteus, Foundations of Stochastic Inventory Theory. Stanford, CA: Stanford University Press, 2002.

[29] P. Cortez, A. Cerdeira, F. Almeida, T. Matos, and J. Reis, "Modeling wine preferences by data mining from physicochemical properties," Decision Support Systems, vol. 47, no. 4, pp. 547–553, 2009.

[30] H. L. Lee, V. Padmanabhan, and S. Whang, "Information distortion in a supply chain: The bullwhip effect," Management Science, vol. 43, no. 4, pp. 546–558, 1997.

APPENDIX A: SAMPLE SOURCE CODE SNIPPETS

A.1 Newsvendor Critical Ratio Reorder Calculator (Python)

```python
import numpy as np
from scipy.stats import norm

def calculate_newsvendor_restock(forecast_mean: float, forecast_sigma: float,
                                 price: float, cogs_ratio: float = 0.40) -> dict:
    # Asymmetric cost parameters
    cost_underage = price * (1.0 - cogs_ratio)  # Gross margin
    cost_overage = price * cogs_ratio           # Lost COGS

    # Critical Ratio calculation
    critical_ratio = cost_underage / (cost_underage + cost_overage)
    z_score = norm.ppf(critical_ratio)          # 0.60 -> +0.2533

    # Recommended order with non-negativity constraint
    recommended_units = max(0, int(np.round(forecast_mean + z_score *
forecast_sigma)))
    safety_buffer = recommended_units - int(np.round(forecast_mean))

    return {"critical_ratio": float(np.round(critical_ratio, 4)),
            "z_score": float(np.round(z_score, 4)),
            "recommended_bake": recommended_units,
            "safety_buffer": safety_buffer}
```

A.2 FastAPI Batch Waste Alert Endpoint (Backend/main.py)

```python
@app.get("/api/v1/waste-alerts", response_model=List[WasteAlertOut])
async def get_waste_alerts(store_id: str, current_user = Depends(get_current_user)):
    alerts = []
    active_items = await db.items.find({"owner_id": current_user.id}).to_list(100)
    for item in active_items:
        projected_waste = compute_projected_waste(item["_id"], store_id)
        if projected_waste >= 8:
            tier = {"severity": "HIGH", "discount_pct": 20,
                    "action": "Apply 20% Markdown Discount"}
        elif projected_waste >= 4:
            tier = {"severity": "MEDIUM", "discount_pct": 10,
                    "action": "Apply 10% Markdown Discount"}
        else:
```

```
            continue
        alerts.append({"item_id": str(item["_id"]), "name": item["name"],
                       "projected_waste": projected_waste, **tier})
    return alerts
```

A.3 LightGBM Residual Feature Engineering Pipeline (ML/features.py)

```python
def build_lag_features(df: pd.DataFrame) -> pd.DataFrame:
    df = df.sort_values(['sku', 'date']).copy()
    # 1-day, 7-day, and 14-day demand lags
    df['demand_lag_1'] = df.groupby('sku')['demand'].shift(1)
    df['demand_lag_7'] = df.groupby('sku')['demand'].shift(7)
    df['demand_lag_14'] = df.groupby('sku')['demand'].shift(14)
    # 7-day moving window statistics
    df['rolling_mean_7'] = df.groupby('sku')['demand'].transform(
        lambda s: s.shift(1).rolling(7).mean()
    )
    df['rolling_std_7'] = df.groupby('sku')['demand'].transform(
        lambda s: s.shift(1).rolling(7).std()
    )
    # Calendar indices
    df['day_of_week'] = df['date'].dt.dayofweek
    df['is_weekend'] = df['day_of_week'].isin([5, 6]).astype(int)
    return df.dropna()
```

APPENDIX B: HARDWARE AND SOFTWARE SPECIFICATIONS

B.1 Hardware Components and Specifications (Table B.1)

Component	Minimum Specification	Development System Specification
Processor (CPU)	4 Cores, 2.0 GHz x86/ARM	AMD Ryzen 7 5800H (8 Cores, 16 Threads @ 3.20 GHz)
System Memory (RAM)	8 GB DDR4	16 GB DDR4 3200 MHz High-Speed RAM
Secondary Storage	256 GB SSD	512 GB PCIe NVMe M.2 Solid State Drive
Mobile Client Device	Android 10 (API 29), 3 GB RAM	OnePlus Nord CE 5G (Android 13, 8 GB RAM, AMOLED)
Network Connectivity	Broadband Internet (10 Mbps)	High-speed Wi-Fi / LTE Internet Connection

Table B.1: Hardware Components and Development Specifications

B.2 Software Environment and Library Specifications (Table B.2)

Software / Library	Version	Operational Role
Python	3.10.11	Core backend and machine learning runtime environment
Facebook Prophet	1.1.5	Stage 1 Generalized Additive Seasonality Decomposition
LightGBM	4.1.0	Stage 2 Pooled Gradient Boosted Tree Residual Regressor
FastAPI	0.104.1	High-performance asynchronous RESTful web microservice
Flutter SDK	3.38.1	Cross-platform mobile application development framework
MongoDB Atlas	6.0 / M0	Distributed cloud document persistence cluster
Docker	24.0.7	Container runtime for automated cloud deployment on Render
Google Gemini	gemini-3-flash	Natural-language operational briefing generation API

Table B.2: Software Environment and Library Specifications

APPENDIX C: FUNCTIONAL MODULE RESPONSIBILITIES

C.1 System Module Responsibilities (Table C.1)

Functional Module	Primary Responsibilities	Technology / Service
Data Ingestion & Cleaning	Extracts POS transaction logs, validates dates, handles closures	Python, Pandas, NumPy
Stage 1 Prophet Decomposition	Models macro-trend, Fourier day-of-week seasonality, holidays	Facebook Prophet (GAM)
Stage 2 LightGBM Regressor	Learns residual non-linear dynamics from engineered demand lags	LightGBM / Scikit-Learn
Newsvendor Optimization	Computes Critical Ratio and optimal stocking fractile (z = +0.2533)	NumPy, SciPy Statistics
Intraday Waste Discounter	Evaluates mid-day stock against sales pace; actuates 20%/10% tags	FastAPI, Python Logic
Grounded LLM Briefing	Synthesizes deterministic JSON context into plain-text morning brief	Google Gemini Flash API
Cloud Persistence	Stores store profiles, users, catalogs, forecasts, and inventory logs	MongoDB Atlas (AWS Cloud)
Mobile Client UI	Presents forecasts, briefings, mid-day check-in, and waste alerts	Flutter 3, Dart, Provider

Table C.1: Functional Module Responsibilities

APPENDIX D: RESEARCH MANUSCRIPT DETAILS

D.1 Research Manuscript Abstract & Target Venues

Title: RestockIQ: Overcoming the Forecast-to-Action Gap in Perishable Food Retail Using Hybrid Gradient Boosting and Newsvendor Optimization

Authors: Yatin Patil, Soumy Raghuvanshi, Chinmay Patil, and Dr. Srikanth Pulipeti

Affiliation: Department of Information Technology, Mukesh Patel School of Technology Management & Engineering, SVKM's NMIMS, Shirpur Campus (M.H.), India.

Target Academic Venues: 2026 International Conference on Information Management and Machine Intelligence (ICIMMI 2026) / IEEE International Conference on Advanced Networks and Telecommunications (ICANET 2027).

Abstract: Perishable retail inventory management suffers from severe food wastage due to volatile customer demand and short product lifecycles. We present RestockIQ, an end-to-end perishable inventory optimization system that bridges machine learning demand forecasting with mathematical inventory decision-making. The architecture couples a two-stage hybrid machine learning pipeline—decomposing multi-scale calendar seasonality with Facebook Prophet and predicting residual errors using a pooled Gradient Boosted Decision Tree—with a stochastic Newsvendor Critical Ratio optimization policy ($CR = 0.60, z^* = +0.2533$). Evaluated across 38 weekly rolling-origin temporal folds on 637 calendar days of French artisan bakery sales, the model achieves 25.48% WAPE (a statistically significant 2.47 percentage point error reduction over Prophet baseline, sign-test $p = 0.0138$). On peak summer folds, the canonical hybrid achieves 22.51% WAPE. A 200-iteration circular permutation test proves weather regressors yield no statistically significant gain ($p = 0.3800$), justifying their exclusion. A 68,883-unit Newsvendor simulation demonstrates €41,635.90 in retail profit at 11.2% food waste.

Follow the given instructions while preparing the report:

1.	Font	Times New Roman.
2	Line spacing	1.5".
3.	Margins (left)	1.5”
4.	Remaining all margins	1”
5.	Header Right corner	Academic Year
6.	Header Left corner	Project Title
7.	Footer: Right Corner	Page Number
8.	Chapter Title	Font size 18 and bold.
9.	Section titles	Font size 14 and bold
10.	Subsection titles	Font size 12 and bold
11.	Figure name (below the figures)	Times new roman , font size 10
12.	Table name (above the table)	Times new roman, font size 10
13.	Report text	Font size 12

Figure D.1: Academic Conference Manuscript Submission Verification